

Program 6.7 Interface for array data type

This `Array.h` interface defines high-level functions for arrays of abstract items: initialize random values, initialize values read from standard input, print the contents, and sort the contents. The item types are defined in a separate interface (see Program 6.9).

```
void randinit(Item [], int);  
void scaninit(Item [], int *);  
void show(Item [], int, int);  
void sort(Item [], int, int);
```

mentation, we need to define the *array* and *item* data type interfaces precisely, then provide implementations.

The interface in Program 6.7 defines examples of high-level operations that we might want to perform on arrays. We want to be able to initialize an array with key values, either randomly or from the standard input; we want to be able to sort the entries (of course!); and we want to be able to print out the contents. These are but a few examples; in a particular application, we might want to define various other operations. With this interface, we can substitute different implementations of the various operations without having to change the client program that uses the interface—`main` in Program 6.6, in this case. The various sort implementations that we are studying can serve as implementations for the `sort` function. Program 6.8 has simple implementations for the other functions. Again, we might wish to substitute other implementations, depending on the application. For example, we might use an implementation of `show` that prints out only part of the array when testing sorts on huge arrays.

In a similar manner, to work with particular types of items and keys, we define their types and declare all the relevant operations on them in an explicit interface, then provide implementations of the operations defined in the item interface. Program 6.9 is an example of such an interface for floating point keys. This code defines the operations that we have been using to compare keys and to exchange items, as well as functions to generate a random key, to read a key from standard input, and to print out the value of a key. Program 6.10 has implementations of these functions for this simple example. Some of the operations are defined as macros in the interface, which approach

Program 6.8 Implementation of array data type

This code provides implementations of the functions defined in Program 6.7, again using the item types and basic functions for processing them that are defined in a separate interface (see Program 6.9).

```
#include <stdio.h>
#include <stdlib.h>
#include "Item.h"
#include "Array.h"
void randinit(Item a[], int N)
{ int i;
  for (i = 0; i < N; i++) a[i] = ITEMrand();
}
void scaninit(Item a[], int *N)
{ int i = 0;
  for (i = 0; i < *N; i++)
    if (ITEMscan(&a[i]) == EOF) break;
  *N = i;
}
void show(itemType a[], int l, int r)
{ int i;
  for (i = l; i <= r; i++) ITEMshow(a[i]);
  printf("\n");
}
```

is generally more efficient; others are C code in the implementation, which approach is generally more flexible.

Programs 6.6 through 6.10 together with any of the sorting routines *as is* in Sections 6.2 through 6.6 provide a test of the sort for floating-point numbers. By providing similar interfaces and implementations for other types of data, we can put our sorts to use for a variety of data—such as long integers (see Exercise 6.49), complex numbers (see Exercise 6.50), or vectors (see Exercise 6.55)—without changing the sort code at all. For more complicated types of items, the interfaces and implementations have to be more complicated, but this implementation work is completely separated from the algorithm-design questions that we have been considering. We can use these same mechanisms with most of the sorting methods that we consider in this

Program 6.9 Sample interface for item data type

The file `Item.h` that is included in Programs 6.6 and 6.8 defines the data representation and associated operations for the items to be sorted. In this example, the items are floating-point keys. We use macros for the `key`, `less`, `exch`, and `compexch` data type operations for use by our sorting programs; we could also define them as functions to be implemented separately, like the three functions `ITEMrand` (return a random key), `ITEMscan` (read a key from standard input) and `ITEMshow` (print the value of a key).

```
typedef double Item;
#define key(A) (A)
#define less(A, B) (key(A) < key(B))
#define exch(A, B) { Item t = A; A = B; B = t; }
#define compexch(A, B) if (less(B, A)) exch(A, B)
Item ITEMrand(void);
int ITEMscan(Item *);
void ITEMshow(Item);
```

chapter and with those that we shall study in Chapters 7 through 9, as well. We consider in detail one important exception in Section 6.10—it leads to a whole family of important sorting algorithms that have to be packaged differently, the subject of Chapter 10.

The approach that we have discussed in this section is a middle road between Program 6.1 and an industrial-strength fully abstract set of implementations complete with error checking, memory management, and even more general capabilities. Packaging issues of this sort are of increasing importance in some modern programming and applications environments. We will necessarily leave some questions unanswered. Our primary purpose is to demonstrate, through the relatively simple mechanisms that we have examined, that the sorting implementations that we are studying are widely applicable.

Exercises

6.49 Write an interface and implementation for the generic item data type (similar to Programs 6.9 and 6.10) to support having the sorting methods sort long integers.

6.50 Write an interface and implementation for the generic item data type to support having the sorting methods sort complex numbers $x + iy$ using the

Program 6.10 Sample implementation for item data type

This code implements the three functions `ITEMrand`, `ITEMscan`, and `ITEMshow` that are declared in Program 6.9. In this code, we refer to the type of the data directly with `double`, use explicit floating-point options in `scanf` and `printf`, and so forth. We include the interface file `Item.h` so that we will discover at compile time any discrepancies between interface and implementation.

```
#include <stdio.h>
#include <stdlib.h>
#include "Item.h"
double ITEMrand(void)
    { return 1.0*rand()/RAND_MAX; }
int ITEMscan(double *x)
    { return scanf("%f", x); }
void ITEMshow(double x)
    { printf("%.5f ", x); }
```

magnitude $\sqrt{x^2 + y^2}$ for the key. *Note:* Ignoring the square root is likely to improve efficiency.

- 6.51 Write an interface that defines a first-class *abstract* data type for generic items (see Section 4.8), and provide an implementation where the items are floating point numbers. Test your program with Programs 6.3 and 6.6.
- ▷ 6.52 Add a function `check` to the array data type in Programs 6.8 and 6.7, which tests whether or not the array is in sorted order.
- 6.53 Add a function `testinit` to the array data type in Programs 6.8 and 6.7, which generates test data according to distributions similar to those illustrated in Figure 6.13. Provide an integer argument for the client to use to specify the distribution.
- 6.54 Change Programs 6.7 and 6.8 to implement an *abstract* data type. (Your implementation should allocate and maintain the array, as in our implementations for stacks and queues in Chapter 3.)
- 6.55 Write an interface and implementation for the generic item data type for use in having the sorting methods sort multidimensional vectors of d integers, putting the vectors in order by first component, those with equal first component in order by second component, those with equal first and second components in order by third component, and so forth.

6.8 Index and Pointer Sorting

The development of a string data type implementation similar to Programs 6.9 and 6.10 is of particular interest, because character strings are widely used as sort keys. Using the C library string-comparison function, we can change the first three lines in Program 6.9 to

```
typedef char *Item;
#define key(A) (A)
#define less(A, B) (strcmp(key(A), key(B)) < 0)
```

to convert it to an interface for strings.

The implementation is more challenging than Program 6.10 because, when working with strings in C, we must be aware of the allocation of memory for the strings. Program 6.11 uses the method that we examined in Chapter 3 (Program 3.17), maintaining a buffer in the data-type implementation. Other options are to allocate mem-

Program 6.11 Data-type implementation for string items

This implementation allows us to use our sorting programs to sort strings. A string is a pointer to a character, so a sort will process an array of pointers to characters, rearranging them so the indicated strings are in alphanumeric order. We statically allocate the storage buffer containing the string characters in this module; dynamic allocation is perhaps more appropriate. The ITEMrand implementation is omitted.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "Item.h"
static char buf[100000];
static int cnt = 0;
int ITEMscan(char **x)
{ int t;
  *x = &buf[cnt];
  t = scanf("%s", *x); cnt += strlen(*x)+1;
  return t;
}
void ITEMshow(char *x)
{ printf("%s ", x); }
```

ory dynamically for each string, or to keep the buffer in the client program. We can use this code (along with the interface described in the previous paragraph) to sort strings of characters, using any of the sort implementations that we have been considering. Because strings are represented as pointers to arrays of characters in C, this program is an example of a *pointer sort*, which we shall consider shortly.

We are faced with memory-management choices of this kind any time that we modularize a program. Who should be responsible for managing the memory corresponding to the concrete realization of some type of object: the client, the data-type implementation, or the system? There is no hard-and-fast answer to this question (some programming-language designers become evangelical when the question is raised). Some modern programming systems (not C) have general mechanisms for dealing with memory management automatically. We will revisit this issue in Chapter 9, when we discuss the implementation of a more sophisticated abstract data type.

One simple approach for sorting without (intermediate) moves of items is to maintain an *index array* with keys in the items accessed only for comparisons. Suppose that the items to be sorted are in an array `data[0], ..., data[N-1]`, and that we do not wish to move them around, for some reason (perhaps they are huge). To get the effect of sorting, we use a *second* array `a` of item indices. We begin by initializing `a[i]` to `i` for `i = 0, ..., N-1`. That is, we begin with `a[0]` having the index of the first data item, `a[1]` having the index of the second data item, and so on. The goal of the sort is to rearrange the index array `a` such that `a[0]` gives the index of the data item with the smallest key, `a[1]` gives the index of the data item with the second smallest key, and so on. Then we can achieve the effect of sorting by accessing the keys through the indices—for example, we could print out the array in sorted order in this way.

Now, we take advantage of the fact that our sort routines access data only through `less` and `exch`. In the item-type interface definition, we specify the type of the items to be sorted to be integers (the indices in `a`) with `typedef int Item`; and leave the exchange as before, but we change `less` to refer to the data through the indices:

```
#define less(A, B) (data[A] < data[B]).
```

For simplicity, this discussion assumes that the data are keys, rather than full items. We can use the same principle for larger, more compli-

cated items, by modifying `less` to access specific keys in the items. The sort routines rearrange the indices in `a`, which carry the information that we need to access the keys. An example of this arrangement, with the same items sorted by two different keys, is shown in Figure 6.14.

This index-array approach to indirection will work in any programming language that supports arrays. Another possibility, especially attractive in C, is to use pointers. For example, defining the data type

```
typedef dataType *Item;
```

and then initializing `a` with

```
for (i = 0; i < N; i++) a[i] = &data[i];
```

and doing comparisons indirectly with

```
#define less(A, B) (*A < *B)
```

is equivalent to using the strategy described in the preceding paragraph. This arrangement is known as a *pointer sort*. The string data-type implementation that we just considered (Program 6.11) is an example of a pointer sort. For sorting an array of fixed-size items, a pointer sort is essentially equivalent to an index sort, but with the address of the array added to each index. But a pointer sort is much more general, because the pointers could point anywhere, and the items being sorted do not need to be fixed in size. As is true in index sorting, if `a` is an array of pointers to keys, then a call to sort will result in the pointers being rearranged such that accessing them sequentially will access the keys in order. We implement comparisons by following pointers; we implement exchanges by exchanging the pointers.

The standard C library sort function `qsort` is a pointer sort (see Program 3.17). The function takes four arguments: the array; the number of items to be sorted; the size of the items; and a pointer to a function that compares two items, given pointers to them. For example, if `Item` is `char*`, then the following code implements a string sort that adheres to our conventions:

```
int compare(void *i, void *j)
{ return strcmp(*(Item *)i, *(Item *)j); }
void sort(Item a[], int l, int r)
{ qsort(a, r-l+1, sizeof(Item), compare); }
```

The underlying algorithm is not specified in the interface, but quicksort (see Chapter 7) is widely used. In Chapter 7 we shall consider many of

0	10	9	Wilson	63
1	4	2	Johnson	86
2	5	1	Jones	87
3	6	0	Smith	90
4	8	4	Washington	84
5	7	8	Thompson	65
6	2	3	Brown	82
7	3	10	Jackson	61
8	9	6	White	76
9	0	5	Adams	86
10	1	7	Black	71

Figure 6.14
Index sorting example

By manipulating indices, rather than the records themselves, we can sort an array simultaneously on several keys. For this sample data that might represent students' names and grades, the second column is the result of an index sort on the name, and the third column is the result of an index sort on the grade. For example, Wilson is last in alphabetic order and has the tenth highest grade, while Adams is first in alphabetic order and has the sixth highest grade.

*A rearrangement of the N distinct nonnegative integers less than N is called a *permutation* in mathematics: an index sort computes a permutation. In mathematics, permutations are normally defined as rearrangements of the integers 1 through N ; we shall use 0 through $N - 1$ to emphasize the direct relationship between permutations and C array indices.*

Program 6.12 Data-type interface for record items

The records have two keys: a string key (for example, a name) in the first field, and an integer key (for example, a grade) in the second field. The comparison `less` is defined as a function, rather than as a macro, so we can change sort keys by changing implementations,

```
struct record { char name[30]; int num; };
typedef struct record* Item;
#define exch(A, B) { Item t = A; A = B; B = t; }
#define compexch(A, B) if (less(B, A)) exch(A, B);
int less(Item, Item);
Item ITEMrand();
int ITEMscan(Item *);
void ITEMshow(Item);
```

the reasons why this is true. We also, in this chapter and in Chapters 7 through 11, develop an understanding of why other methods might be more appropriate for some specific applications, and we explore approaches for speeding up the computation when the sort time is a critical factor in an application.

The primary reason to use indices or pointers is to avoid intruding on the data being sorted. We can “sort” a file even if read-only access is all that is available. Moreover, with multiple index or pointer arrays, we can sort one file on multiple keys (see Figure 6.14). This flexibility to manipulate the data without actually changing them is useful in many applications.

A second reason for manipulating indices is that we can avoid the cost of moving full records. The cost savings is significant for files with large records (and small keys), because the comparison needs to access just a small part of the record, and most of the record is not even touched during the sort. The indirect approach makes the cost of an exchange roughly equal to the cost of a comparison for general situations involving arbitrarily large records (at the cost of the extra space for the indices or pointers). Indeed, if the keys are long, the exchanges might even wind up being less costly than the comparisons. When we estimate the running times of methods that sort files of integers, we are often making the assumption that the costs of

Program 6.13 Data-type implementation for record items

These implementations of the `ITEMscan` and `ITEMshow` functions for records operate in a manner similar to the string data-type implementation of Program 6.11, in that they allocate and maintain the memory for the records. We keep the implementation of `less` in a separate file, so that we can substitute different implementations, and therefore change sort keys, without changing any other code.

```

struct record data[maxN];
int Nrecs = 0;
int ITEMscan(struct record **x)
{
    *x = &data[Nrecs];
    return scanf("%30s %d\n",
                data[Nrecs].name, &data[Nrecs++].num);
}
void ITEMshow(struct record *x)
{ printf("%3d %-30s\n", x->num, x->name); }
```

comparisons and exchanges are not much different. Conclusions based on this assumption are likely to apply to a broad class of applications, if we use pointer or index sorts.

In typical applications, the pointers are used to access records that may contain several possible keys. For example, records consisting of students' names and grades or people's names and ages:

```
struct record { char[30] name; int num; }
```

Programs 6.12 and 6.13 provide an example of a pointer sort interface and implementation that can allow us to sort them using either of the fields as key. We use an array of pointers to records, and declare `less` as a function, rather than a macro. Then we can provide different implementations of `less` for different sort applications. For example, if we compile Program 6.13 together with a file containing

```

#include "Item.h"
int less(Item a, Item b)
{ return a->num < b->num; }
```

then we get a data type for the items for which any of our sort implementations will do a pointer sort on the integer field. Alternatively,

we might choose to use the string field of the records for the sort keys. If we compile Program 6.13 together with a file containing

```
#include <string.h>
#include "Item.h"
int less(Item a, Item b)
{ return strcmp(a->name, b->name) < 0; }
```

then we get a data type for the items for which any of our sort implementations will do a pointer sort on the string field.

For many applications, the data never need to be rearranged physically to match the order indicated by the indices, and we can simply access them in order using the index array. If this approach is not satisfactory for some reason, we are led to a classic programming exercise: How do we rearrange a file that has been sorted with an index sort? The code

```
for (i = 0; i < N; i++) datasorted[i] = data[a[i]];
```

is trivial, but requires extra memory sufficient for another copy of the array. What about the situation when there is not enough room for another copy of the file? We cannot blindly set `data[i] = data[a[i]]`, because that would overwrite the previous value of `data[i]`, perhaps prematurely.

Figure 6.15 illustrates how we can solve this problem, still using a single pass through the file. To move the first element where it belongs, we move the element at that position to where it belongs, and so forth. Continuing this reasoning, we eventually find an element to move to the first position, at which point we have shifted a cycle of elements into position. Then, we move to the second element and perform the same operation for its cycle, and so forth (any elements that we encounter that are already in position (`a[i]=i`) are on a cycle of length 1 and are not moved).

Specifically, for each value of `i`, we save the value of `data[i]` and initialize an index variable `k` to `i`. Now, we think of a hole in the array at `i`, and seek an element to fill the hole. That element is `data[a[k]]`—in other words, the assignment `data[k] = data[a[k]]` moves the hole to `a[k]`. Now the hole is at `data[a[k]]`, so we set `k` to `a[k]`. Iterating, we eventually get to a situation where the hole needs to be filled by `data[i]`, which we have saved. When we move an element into position we update the `a` array to so indicate. Any

Program 6.14 In-place sort

The array `data[0], ..., data[N-1]` is to be rearranged in place as directed by the index array `a[0], ..., a[N-1]`. Any element with `a[i] == i` is in place and does not need to be touched again. Otherwise, save `data[i]` as `v` and work through the cycle `a[i], a[a[i]], a[a[a[i]]]`, and so on, until reaching the index `i` again. We follow the process again for the next element which is not in place, and continue in this manner, ultimately rearranging the entire file, moving each record only once.

```
insitu(dataType data[], int a[], int N)
{
    int i, j, k;
    for (i = 0; i < N; i++)
    {
        dataType v = data[i];
        for (k = i; a[k] != i; k = a[j], a[j] = j)
        {
            j = k; data[k] = data[a[k]];
        }
        data[k] = v; a[k] = k;
    }
}
```

element in position has `a[i]` equal to `i`, and the process just outlined is a no-op in that case. Continuing through the array, starting a new cycle each time that we encounter an element not yet moved, we move every element at most once. Program 6.14 is an implementation of this process.

This process is called *in situ permutation*, or *in-place rearrangement* of the file. Again, although the algorithm is interesting, it is unnecessary in many applications, because accessing the data indirectly often suffices. Also, if the records are huge relative to their number, the most efficient option may be simply to rearrange them with a conventional selection sort (see Property 6.5).

Indirect sorting requires extra space for the index or pointer array and extra time for the indirect comparisons. In many applications, these costs are a small price to pay for the flexibility of not having to move the data at all. For files consisting of large records, we will almost always choose to use an indirect sort, and for many applications, we will find that it is not necessary to move the data at all. In this book, we normally will access data directly. In a few applications, however, we

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
A S O R T I N G E X A M P L E
0 10 8 14 7 5 13 11 6 2 12 3 1 4 9
```

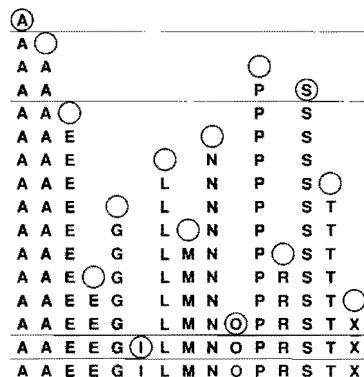


Figure 6.15
In-place sort

To rearrange an array in place, we move from left to right, moving elements that need to be moved in cycles. Here, there are four cycles: The first and last are single-element degenerate cases. The second cycle starts at 1. The S goes into a temporary variable, leaving a hole at 1. Moving the second A there leaves a hole at 10. This hole is filled by P, which leaves a hole at 12. That hole is to be filled by the element at position 1, so the reserved S goes into that hole, completing the cycle 1 10 12 that puts those elements in position. Similarly, the cycle 2 8 6 13 4 7 11 3 14 9 completes the sort.

do use pointers or index arrays to avoid data movement, for precisely the reasons mentioned here.

Exercises

- 6.56 Give an implementation of a data type for items where the items are records, rather than pointers to records. This arrangement might be preferable to Programs 6.12 and 6.13 for small records. (Remember that C supports structure assignment.)
- 6.57 Show how to use `qsort` to solve the sorting problem that is addressed in Programs 6.12 and 6.13.
- ▷ 6.58 Give the index array that results when the keys `EASYQUESTION` are index sorted.
- ▷ 6.59 Give the sequence of data moves required to permute the keys `EASYQUESTION` in place after an index sort (see Exercise 6.58).
- 6.60 Describe a permutation of size N (a set of values for the array `a`) that maximizes the number of times that `a[i] != i` during Program 6.14.
- 6.61 Prove that we are guaranteed to return to the key with which we started when moving keys and leaving holes in Program 6.14.
- 6.62 Implement a program like Program 6.14 corresponding to a pointer sort. Assume that the pointers point into an array of N records, of type `Item`.

6.9 Sorting of Linked Lists

As we know from Chapter 3, arrays and linked lists provide two of the most basic ways to structure data, and we considered an implementation of insertion sort for linked lists as a list-processing example in Section 3.4 (Program 3.11). The sort implementations that we have considered to this point all assume that the data to be sorted is in an array, and are not directly applicable if we are working within a system that uses linked lists to organize data. In some cases, the *algorithms* may be useful, but only if they process data in the essentially sequential manner that we can support efficiently for linked lists.

Program 6.15 gives an interface, which is similar to Program 6.7, for a *linked-list* data type. With Program 6.15, the driver program corresponding to Program 6.6 is a one-liner:

```
main(int argc, char *argv[])
{ show(sort(init(atoi(argv[1])))); }
```

Most of the work (including allocation of memory) is left to the linked-list and sort implementations. As we did with our array driver, we

Program 6.15 Linked-list-type interface definition

This interface for linked lists can be contrasted with the one for arrays in Program 6.7. The `init` function builds the list, including storage allocation. The `show` function prints out the keys in the list. Sorting programs use `less` to compare items and manipulate pointers to rearrange the items. We do not specify here whether or not lists have head nodes.

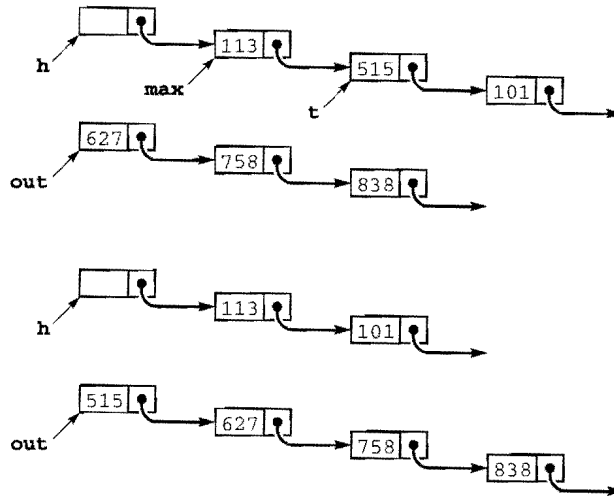
```
typedef struct node *link;
struct node { Item item; link next; };
link NEW(Item, link);
link init(int);
void show(link);
link sort(link);
```

want to initialize the list (either from standard input or with random values), to show the contents of the list, and, of course, to sort it. As usual, we use an `Item` for the data type of the items being sorted, just as we did in Section 6.7. The code to implement the routines for this interface is standard for linked lists of the kind that we examined in detail in Chapter 3, and left as an exercise.

There is a ground rule for manipulating linked structures that is critical in many applications, but is not evident from this code. In a more complex environment, it could be the case that pointers to the list nodes that we are manipulating are maintained by other parts of the applications system (i.e., they are in multilists). The possibility that nodes could be referenced through pointers that are maintained outside the sort means that our programs should *change only links in nodes, and should not alter keys or other information*. For example, when we want to do an exchange, it would seem simplest just to exchange items (as we did when sorting arrays). But then any reference to either node with some other link would find the value changed, and probably will not have the desired effect. We need to change the links themselves such that the nodes appear in sorted order when the list is traversed via the links we have access to, without affecting their order when accessed via any other links. Doing so makes the implementations more difficult, but usually is necessary.

Figure 6.16
Linked-list selection sort

This diagram depicts one step of selection sort for linked lists. We maintain an input list, pointed to by `h->next`, and an output list, pointed to by `out` (top). We scan through the input list to make `max` point to the node before (and `t` point to) the node containing the maximum item. These are the pointers we need to remove `t` from the input list (reducing its length by 1) and put it at the front of the output list (increasing its length by 1), keeping the output list in order (bottom). Iterating, we eventually exhaust the input list and have the nodes in order in the output list.



We can adapt insertion, selection, and bubble sort to linked-list implementations, although each one presents amusing challenges. Selection sort is straightforward: We maintain an input list (which initially has the data) and an output list (which collects the sorted result), and simply scan through the list to find the maximum element in the input list, remove it from the list, and add it to the front of the output list (see Figure 6.16). Implementing this operation is a simple exercise in linked-list manipulation, and is a useful method for sorting short lists. An implementation is given in Program 6.16. We leave the other methods for exercises.

In some list-processing situations, we may not need to explicitly implement a sort at all. For example, we could choose to keep the list in order at all times, inserting new nodes into the list as in insertion sort. This approach comes at little extra cost if insertions are relatively rare or the list is small, and in certain other situations. For example, we might need to scan the whole list for some reason before inserting new nodes (perhaps to check for duplicates). We shall discuss an algorithm that uses ordered linked lists in Chapter 14, and we shall see numerous data structures that gain efficiency from order in the data in Chapters 12 and 14.

Program 6.16 Linked-list selection sort

Selection sort of a linked list is straightforward, but differs slightly from the array version because it is easier to insert at the front of a list. We maintain an input list (pointed to by `h->next`), and an output list (pointed to by `out`). While it is nonempty, we scan the input list to find the maximum remaining element, then remove that element from the input list and insert it at the front of the output list. This implementation uses an auxiliary routine `findmax`, which returns a link to the node whose link points to the maximum element on a list (see Exercise 3.34).

```
link listselection(link h)
{ link max, t, out = NULL;
  while (h->next != NULL)
  {
    max = findmax(h);
    t = max->next; max->next = t->next;
    t->next = out; out = t;
  }
  h->next = out;
  return(h);
}
```

Exercises

- ▷ 6.63 Give the contents of the input list and output list as Program 6.16 is used for the keys `ASORTINGEXAMPLE`.
- 6.64 Provide an implementation for the linked-list interface given in Program 6.15.
- 6.65 Implement a performance-driver client program for linked-list sorts (see Exercise 6.9).
- 6.66 Implement bubble sort for a linked list. *Caution:* exchanging two adjacent elements on a linked list is more difficult than it seems at first.
- ▷ 6.67 Package the insertion-sort code in Program 3.11 such that it has the same functionality as Program 6.16.
- 6.68 The insertion-sort method used in Program 3.11 makes the linked-list insertion sort run significantly slower than the array version for some input files. Describe one such file, and explain the problem.
- 6.69 Implement a linked-list version of shellsort that does not use significantly more time or space than the array version for large random files. *Hint:* Use bubble sort.

- 6.70 Implement an ADT for *sequences*, which allows us to use a single driver program to debug both linked-list and array sort implementations, for example with the following code:

```
#include "Item.h"
#include "SEQ.h"
main(int argc, char *argv[])
{ int N = atoi(argv[1]), sw = atoi(argv[2]);
  if (sw) SEQrandinit(N); else SEQscaninit(&N);
  SEQsort();
  SEQshow();
}
```

That is, client programs can create a sequence with N items (either generated randomly or filled from standard input), sort the sequence, or show its contents. Provide one implementation that uses an array representation and another that uses a linked-list representation. Use selection sort.

- 6.71 Extend your implementation from Exercise 6.70 such that it is a first-class ADT.

6.10 Key-Indexed Counting

A number of sorting algorithms gain efficiency by taking advantage of special properties of keys. For example, consider the following problem: Sort a file of N items whose keys are distinct integers between 0 and $N-1$. We can solve this problem immediately, using a temporary array b , with the statement

```
for (i = 0; i < N; i++) b[key(a[i])] = a[i];
```

That is, we sort by using the keys as *indices*, rather than as abstract items that are compared. In this section, we consider an elementary method that uses key indexing in this way to sort efficiently when the keys are integers in a small range.

If all the keys are 0, sorting is trivial, but now suppose that there are two distinct key values 0 and 1. Such a sorting problem might arise when we want to separate out the items in a file that satisfy some (perhaps complicated) acceptance test: we take the key 0 to mean “accept” and the key 1 to mean “reject.” One way to proceed is to count the number of 0s, then to make a second pass through the input a to distribute its items to the temporary array b , using an array of two counters, as follows. We start with 0 in $\text{cnt}[0]$ and the number of 0 keys in the file $\text{cnt}[1]$, to indicate that there are no keys that are less than 0 and $\text{cnt}[1]$ keys that are less than 1 in the file. Clearly,

we can fill in the b array by putting 0s at the beginning (starting at $b[\text{cnt}[0]]$, or $b[0]$) and 1s starting at $b[\text{cnt}[1]]$. That is, the code

```
for (i = 0; i < N; i++) b[cnt[a[i]]++] = a[i];
```

serves to distribute the items from a to b . Again, we get a fast sort by using the keys as indices (to pick between $\text{cnt}[0]$ and $\text{cnt}[1]$). We could use an `if` statement to choose between the two counters in this simple case, but the approach of using keys as indices generalizes immediately to handle more than two key values (more than two counters).

Specifically, a more realistic problem in the same spirit is this: Sort a file of N items whose keys are integers between 0 and $M - 1$. We can extend the basic method in the previous paragraph to an algorithm called *key-indexed counting*, which solves this problem effectively if M is not too large. Just as with two key values, the idea is to count the number of keys with each value, and then to use the counts to move the items into position on a second pass through the file. First, we count the number of keys of each value: then, we compute partial sums to get counts of the number of keys less than or equal to each value. Then, again just as we did when we had two key values, we use these counts as indices for the purpose of distributing the keys. For each key, we view its associated count as an index pointing to the end of the block of keys with the same value, use the index to distribute the key into b , and decrement. The critical factor that makes this algorithm efficient is that we do not need to go through a chain of `if` statements to determine which counter to access—using the key as index, we immediately find the right one. This process is illustrated in Figure 6.17. An implementation is given in Program 6.17.

Property 6.12 *Key-indexed counting is a linear-time sort, provided that the range of distinct key values is within a constant factor of the file size.*

Each item is moved twice, once for the distribution and once to be moved back to the original array; and each key is referenced twice, once to do the counts and once to do the distribution. The two other `for` loops in the algorithm involve building the counts, and will contribute insignificantly to the running time unless the number of counts becomes significantly larger than the file size. ■

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	3	3	0	1	1	0	3	0	2	0	1	1	2	0

0	1	2	3
0	6	4	2
0	6	10	12

0	0													
3	0												3	
3	0												3	3
0	0	0											3	3
1	0	0				1							3	3
1	0	0				1	1						3	3
0	0	0	0			1	1						3	3
3	0	0	0			1	1						3	3
0	0	0	0	0		1	1						3	3
2	0	0	0	0		1	1		2				3	3
0	0	0	0	0	0	1	1		2				3	3
1	0	0	0	0	0	1	1	1	2				3	3
1	0	0	0	0	0	1	1	1	1	2			3	3
2	0	0	0	0	0	1	1	1	1	2	2		3	3
0	0	0	0	0	0	1	1	1	1	2	2	3	3	3
0	0	0	0	0	0	1	1	1	1	2	2	3	3	3

Figure 6.17
Sorting by key-indexed counting.

First, we determine how many keys of each value there are in the file: In this example there are six 0s, four 1s, two 2s, and three 3s. Then, we take partial sums to find the number of keys less than each key: 0 keys are less than 0, 6 keys are less than 1, 10 keys are less than 2, and 12 keys are less than 3 (table in middle). Then, we use the partial sums as indices in placing the keys into position: The 0 at the beginning of the file is put into location 0; we then increment the pointer corresponding to 0, to point to where the next 0 should go. Then, the 3 from the next position on the left in the file is put into location 12 (since there are 12 keys less than 3); its corresponding count is incremented; and so forth.

Program 6.17 Key-indexed counting

The first for loop initializes the counts to 0; the second for loop sets the second counter to the number of 0s, the third counter to the number of 1s, and so forth. Then, the third for loop simply adds these numbers to produce counts of the number of keys less than or equal to the one corresponding to the count. These numbers now give the indices of the end of the part of the file where keys belong. The fourth for loop moves the keys into an auxiliary array *b* according to these indices, and the final loop moves the sorted file back into *a*. The keys must be integers less than *M* for this code to work, although we can easily modify it to extract such keys from more complex items (see Exercise 6.75).

```
void distcount(int a[], int l, int r)
{
    int i, j, cnt[M];
    int b[maxN];
    for (j = 0; j < M; j++) cnt[j] = 0;
    for (i = l; i <= r; i++) cnt[a[i]+1]++;
    for (j = 1; j < M; j++) cnt[j] += cnt[j-1];
    for (i = l; i <= r; i++) b[cnt[a[i]]++] = a[i];
    for (i = l; i <= r; i++) a[i] = b[i-l];
}
```

If huge files are to be sorted, the auxiliary array *b* can present memory-allocation problems. We can modify Program 6.17 to complete the sort in place (avoiding the need for an auxiliary array), using a method similar to that used in Program 6.14. This operation is closely related to the basic methods that we shall be discussing in Chapters 7 and 10, so we defer it to Exercises 12.16 and 12.17 in Section 12.3. As we shall see in Chapter 12, this space savings comes at the cost of the stability property of the algorithm, and thus limits the algorithm's utility because applications involving large numbers of duplicate keys often have other associated keys, whose relative order should be preserved. We shall see a particularly important example of such an application in Chapter 10.

Exercises

- 6.72 Give a specialized version of key-indexed counting for sorting files where elements can take on only one of three values (*a*, *b*, or *c*).

6.73 Suppose that we use insertion sort on a randomly ordered file where elements have only one of three values. Is the running time linear, quadratic, or something in between?

▷ **6.74** Show how key-indexed counting sorts the file A B R A C A D A B R A.

6.75 Implement key-indexed counting for items that are potentially large records with integer keys from a small range.

6.76 Implement key-indexed counting as a pointer sort.

CHAPTER SEVEN

Quicksort

THE SUBJECT OF this chapter is the sorting algorithm that is probably used more widely than any other, *quicksort*. The basic algorithm was invented in 1960 by C. A. R. Hoare, and it has been studied by many people since that time (*see reference section*). Quicksort is popular because it is not difficult to implement, works well for a variety of different kinds of input data, and consumes fewer resources than any other sorting method in many situations.

The quicksort algorithm has the desirable features that it is in-place (uses only a small auxiliary stack), requires requires time only proportional to $N \log N$ on the average to sort N items, and has an extremely short inner loop. Its drawbacks are that it is not stable, takes about N^2 operations in the worst case, and is fragile in the sense that a simple mistake in the implementation can go unnoticed and can cause it to perform badly for some files.

The performance of quicksort is well understood. The algorithm has been subjected to a thorough mathematical analysis, and we can make precise statements about its performance. The analysis has been verified by extensive empirical experience, and the algorithm has been refined to the point where it is the method of choice in a broad variety of practical sorting applications. It is therefore worthwhile for us to look more carefully than for other algorithms at ways of implementing quicksort efficiently. Similar implementation techniques are appropriate for other algorithms; with quicksort, we can use them with confidence, because we know precisely how they will affect performance.

It is tempting to try to develop ways to improve quicksort: A faster sorting algorithm is computer science's "better mousetrap," and quicksort is a venerable method that seems to invite tinkering. Almost from the moment Hoare first published the algorithm, improved versions have been appearing in the literature. Many ideas have been tried and analyzed, but it is easy to be deceived, because the algorithm is so well balanced that the effects of improvements in one part of the program can be more than offset by the effects of bad performance in another part of the program. We examine in detail three modifications that do improve quicksort substantially.

A carefully tuned version of quicksort is likely to run significantly faster on most computers than will any other sorting method, and quicksort is widely used as a library sort utility and for other serious sorting applications. Indeed, the standard C library's sort is called `qsort`, since quicksort is typically the underlying algorithm used in implementations. However, the running time of quicksort depends on the input, ranging from linear to quadratic in the number of items to be sorted, and people are sometimes surprised by undesirable and unexpected effects for some inputs, particularly in highly tuned versions of the algorithm. If an application does not justify the work required to be sure that a quicksort implementation is not flawed, shellsort might well be a safer choice that will perform well for less implementation investment. For huge files, however, quicksort is likely to run five to ten times as fast as shellsort, and it can adapt to be even more efficient for other types of files that might occur in practice.

7.1 The Basic Algorithm

Quicksort is a divide-and-conquer method for sorting. It works by *partitioning* an array into two parts, then sorting the parts independently. As we shall see, the precise position of the partition depends on the initial order of the elements in the input file. The crux of the method is the partitioning process, which rearranges the array to make the following three conditions hold:

- The element $a[i]$ is in its final place in the array for some i .
- None of the elements in $a[1], \dots, a[i-1]$ is greater than $a[i]$.
- None of the elements in $a[i+1], \dots, a[r]$ is less than $a[i]$.

Program 7.1 Quicksort

If the array has one or fewer elements, do nothing. Otherwise, the array is processed by a **partition** procedure (see Program 7.2), which puts $a[i]$ into position for some i between l and r inclusive, and rearranges the other elements such that the recursive calls properly finish the sort.

```
int partition(Item a[], int l, int r);
void quicksort(Item a[], int l, int r)
{ int i;
  if (r <= l) return;
  i = partition(a, l, r);
  quicksort(a, l, i-1);
  quicksort(a, i+1, r);
}
```

We achieve a complete sort by partitioning, then recursively applying the method to the subfiles, as depicted in Figure 7.1. Because the partitioning process always puts at least one element into position, a formal proof by induction that the recursive method constitutes a proper sort is not difficult to develop. Program 7.1 is a recursive program that implements this idea.

We use the following general strategy to implement partitioning. First, we arbitrarily choose $a[r]$ to be the *partitioning element*—the one that will go into its final position. Next, we scan from the left end of the array until we find an element greater than the partitioning element, and we scan from the right end of the array until we find an element less than the partitioning element. The two elements that stopped the scans are obviously out of place in the final partitioned array, so we exchange them. Continuing in this way, we ensure that no array elements to the left of the left pointer are greater than the partitioning element, and no array elements to the right of the right pointer are less than the partitioning element, as depicted in the following diagram:

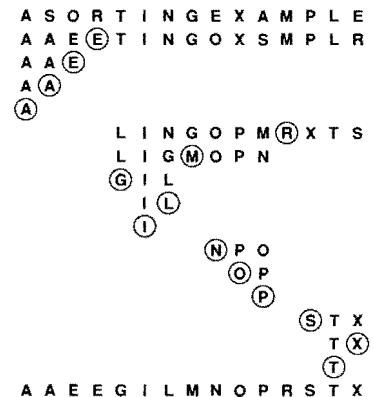


Figure 7.1
Quicksort example

Quicksort is a recursive partitioning process: We partition a file by putting some element (the partitioning element) in place, and rearranging the array such that smaller elements are to the left of the partitioning element and larger elements to its right. Then, we sort the left and right parts of the array recursively. Each line in this diagram depicts the result of partitioning the displayed subfile using the circled element. The end result is a fully sorted file.

```

A S O R T I N G E X A M P L E ⑤
A S
A A           A M P L
                S M P L E
      O
A A E           E X
                O X S M P L E
      R
    E R T I N G
A A E ⑤ T I N G O X S M P L R

```

Figure 7.2
Quicksort partitioning

Quicksort partitioning begins with the (arbitrary) choice of a partitioning element. Program 7.2 uses the rightmost element E. Then, it scans from the left over smaller elements and from the right over larger elements, exchanges the elements that stop the scans, continuing until the scan pointers meet. First, we scan from the left and stop at the S, then we scan from the right and stop at the A, and then we exchange the S and the A. Next, we continue the scan from the left until we stop at the O, and continue the scan from the right until we stop at the E, then exchange the O and the E. Next, our scanning pointers cross: We continue the scan from the left until we stop at the R, then continue the scan from the right (past the R) until we stop at the E. To finish the process, we exchange the partitioning element (the E at the right) with the R.

Here, *v* refers to the value of the partitioning element, *i* to the left pointer, and *j* to the right pointer. As indicated in this diagram, it is best to stop the left scan for elements greater than *or equal* to the partitioning element and the right scan for elements less than *or equal* to the partitioning element, even though this policy might seem to create unnecessary exchanges involving elements equal to the partitioning element (we shall examine the reasons for this policy later in this section). When the scan pointers cross, all that we need to do to complete the partitioning process is to exchange a [*r*] with the leftmost element of the right subfile (the element pointed to by the left pointer). Program 7.2 is an implementation of this process, and Figures 7.2 and 7.3 depict examples.

The inner loop of quicksort increments a pointer and compares an array element against a fixed value. This simplicity is what makes quicksort quick: It is hard to envision a shorter inner loop in a sorting algorithm.

Program 7.2 uses an explicit test to stop the scan if the partitioning element is the smallest element in the array. It might be worthwhile to use a sentinel to avoid this test: The inner loop of quicksort is so small that this one superfluous test could have a noticeable effect on performance. A sentinel is not needed for this implementation when the partitioning element is the largest element in the file, because the partitioning element itself is at the right end of the array to stop the scan. Other implementations of partitioning discussed later in this section and elsewhere in this chapter do not necessarily stop the scan on keys equal to the partitioning element—we might need to add a test to stop the pointer from running off the right end of the array in such an implementation. On the other hand, the improvement to quicksort that we discuss in Section 7.5 has the side benefit of needing neither the test nor a sentinel at either end.

The partitioning process is not stable, because any key might be moved past a large number of keys equal to it (which have not even been examined yet) during any exchange. No easy way to make an array-based quicksort stable is known.

The partitioning procedure must be implemented carefully. Specifically, the most straightforward way to guarantee that the recursive program terminates is that it (i) does not call itself for files of size 1 or less; and (ii) calls itself for *only* files that are strictly smaller

Program 7.2 Partitioning

The variable v holds the value of the partitioning element $a[r]$, and i and j are the left and right scan pointers, respectively. The partitioning loop increments i and decrements j , while maintaining the invariant property that no elements to the left of i are greater than v and no elements to the right of j are smaller than v . Once the pointers meet, we complete the partitioning by exchanging $a[i]$ and $a[r]$, which puts v into $a[i]$, with no larger elements to v 's right and no smaller elements to its left.

The partitioning loop is implemented as an infinite loop, with a `break` when the pointers cross. The test `j == 1` protects against the case that the partitioning element is the smallest element in the file.

```
int partition(Item a[], int l, int r)
{ int i = l-1, j = r; Item v = a[r];
  for (;;)
  {
    while (less(a[++i], v)) ;
    while (less(v, a[--j])) if (j == 1) break;
    if (i >= j) break;
    exch(a[i], a[j]);
  }
  exch(a[i], a[r]);
  return i;
}
```

than given as input. These policies may seem obvious, but it is easy to overlook a property of the input that can lead to a spectacular failure. For instance, a common mistake in implementing quicksort is not ensuring that one element is always put into position, then falling into an infinite recursive loop when the partitioning element happens to be the largest or smallest element in the file.

When duplicate keys are present in the file, the pointer crossing is subtle. We could improve the partitioning process slightly by terminating the scans when $j < i$, and then using j , rather than $i-1$, to delimit the right end of the left subfile for the first recursive call. Letting the loop iterate one more time in this case is an improvement, because, whenever the scanning loops terminate with j and i referring to the same element, we end up with *two* elements in their final posi-

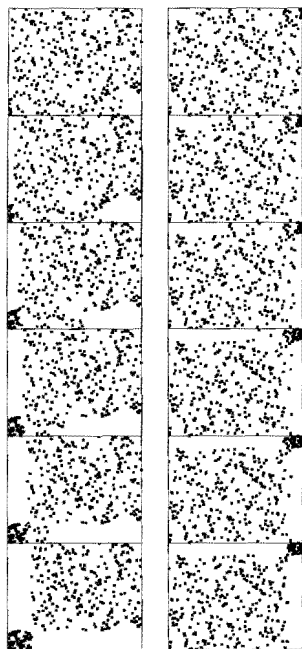


Figure 7.3
Dynamic characteristics of
quicksort partitioning

The partitioning process divides a file into two subfiles that can be sorted independently. None of the elements to the left of the left scan pointer is larger, so there are no dots above and to its left; and none of the elements to the right of the right scan pointer is smaller, so there are no dots below and to its right. As shown in these two examples, partitioning a random array divides it into two smaller random arrays, with one element (the partitioning element) ending up on the diagonal.

tions: the element that stopped both scans, which must therefore be equal to the partitioning element, and the partitioning element itself. This case would occur, for example, if R were E in Figure 7.2. This change is probably worth making, because, in this particular case, the program as given leaves a record with a key equal to the partitioning key in $a[r]$, and that makes the first partition in the call `quicksort(a, i+1, r)` degenerate, because its rightmost key is its smallest. Separating partitioning out as in Programs 7.1 and 7.2 makes quicksort a bit easier to understand, however, so we refer to the combination as the basic quicksort algorithm. If significant numbers of duplicate keys might be present, other factors come into play. We consider them next.

There are three basic strategies that we could adopt with respect to keys equal to the partitioning element: have both pointers stop on such keys (as in Program 7.2); have one pointer stop and the other scan over them; or have both pointers scan over them. The question of which of these strategies is best has been studied in detail mathematically, and results show that it is best to have both pointers stop, primarily because this strategy tends to balance the partitions in the presence of many duplicate keys, whereas the other two can lead to badly unbalanced partitions for some files. We also consider a slightly more complicated and much more effective method for dealing with duplicate keys in Section 7.6.

Ultimately, the efficiency of the sort depends on how well the partitioning divides the file, which in turn depends on the value of the partitioning element. Figure 7.2 shows that partitioning divides a large randomly ordered file into two smaller randomly ordered files, but that the actual split could be anywhere in the file. We would prefer to choose an element that would split the file near the middle, but we do not have the necessary information to do so. If the file is randomly ordered, choosing $a[r]$ as the partitioning element is the same as choosing any other specific element, and will give us a split near the middle *on the average*. In Section 7.4 we consider the analysis of the algorithm that allows us to see how this choice compares to the ideal choice. In Section 7.5 we see how the analysis guides us in considering choices of the partitioning element that make the algorithm more efficient.

Exercises

- ▷ 7.1 Show, in the style of the example given here, how quicksort sorts the file `EASYQUESTION`.
- 7.2 Show how the file `1 0 0 1 1 1 0 0 0 0 0 1 0 1 0 0` is partitioned, using both Program 7.2 and the minor modifications suggested in the text.
- 7.3 Implement partitioning without using a `break` statement or a `goto` statement.
- 7.4 Develop a stable quicksort for linked lists.
- 7.5 What is the maximum number of times during the execution of quicksort that the largest element can be moved, for a file of N elements?

7.2 Performance Characteristics of Quicksort

Despite its many assets, the basic quicksort program has the definite liability that it is extremely inefficient on some simple files that can arise in practice. For example, if it is called with a file of size N that is already sorted, then all the partitions will be degenerate, and the program will call itself N times, removing just one element for each call.

Property 7.1 *Quicksort uses about $N^2/2$ comparisons in the worst case.*

By the argument just given, the number of comparisons used for a file that is already in order is

$$N + (N - 1) + (N - 2) + \dots + 2 + 1 = (N + 1)N/2.$$

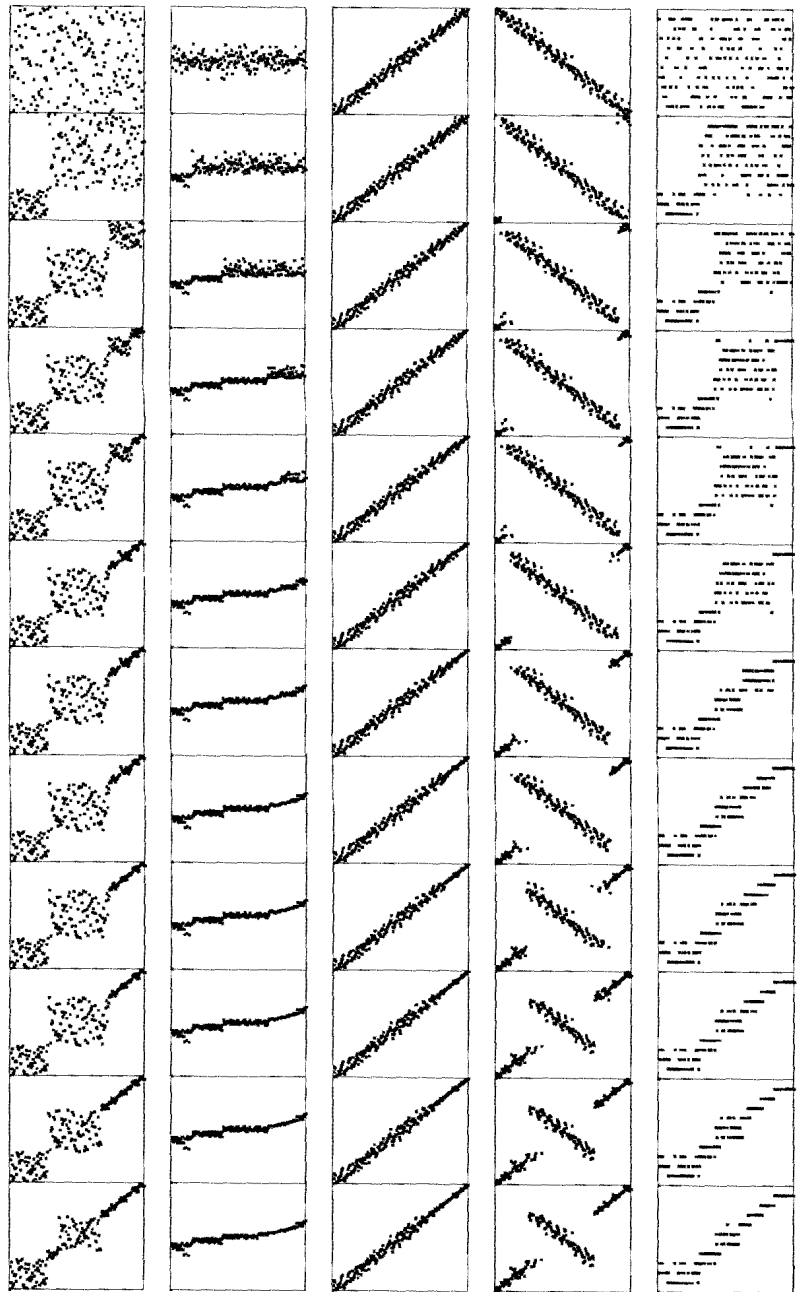
All the partitions are also degenerate for files in reverse order, as well as for other kinds of files that are less likely to occur in practice (see Exercise 7.6). ■

This behavior means not only that the time required will be about $N^2/2$, but also that the space required to handle the recursion will be about N (see Section 7.3), which is unacceptable for large files. Fortunately, there are relatively easy ways to reduce drastically the likelihood that this worst case will occur in typical applications of the program.

The best case for quicksort is when each partitioning stage divides the file exactly in half. This circumstance would make the number of

Figure 7.4
**Dynamic characteristics of
quicksort on various types
of files**

The choice of an arbitrary partitioning element in quicksort results in differing partitioning scenarios for different files. These diagrams illustrate the initial portions of scenarios for files that are random, Gaussian, nearly ordered, nearly reverse ordered, and randomly ordered with 10 distinct key values (left to right), using a relatively large value of the cutoff for small subfiles. Elements not involved in partitioning end up close to the diagonal, leaving an array that could be handled easily by insertion sort. The nearly ordered files require an excessive number of partitions.



comparisons used by quicksort satisfy the divide-and-conquer recurrence

$$C_N = 2C_{N/2} + N.$$

The $2C_{N/2}$ covers the cost of sorting the two subfiles; the N is the cost of examining each element, using one partitioning pointer or the other. From Chapter 5, we know that this recurrence has the solution

$$C_N \approx N \lg N.$$

Although things do not always go this well, it is true that the partition falls in the middle *on the average*. Taking into account the precise probability of each partition position makes the recurrence more complicated and more difficult to solve, but the final result is similar.

Property 7.2 *Quicksort uses about $2N \ln N$ comparisons on the average.*

The precise recurrence formula for the number of comparisons used by quicksort for N randomly ordered distinct elements is

$$C_N = N + 1 + \frac{1}{N} \sum_{1 \leq k \leq N} (C_{k-1} + C_{N-k}) \quad \text{for } N \geq 2,$$

with $C_1 = C_0 = 0$. The $N + 1$ term covers the cost of comparing the partitioning element with each of the others (two extra for where the pointers cross); the rest comes from the observation that each element k is likely to be the partitioning element with probability $1/k$, after which we are left with random files of size $k - 1$ and $N - k$.

Although it looks rather complicated, this recurrence is actually easy to solve, in three steps. First, $C_0 + C_1 + \cdots + C_{N-1}$ is the same as $C_{N-1} + C_{N-2} + \cdots + C_0$, so we have

$$C_N = N + 1 + \frac{2}{N} \sum_{1 \leq k \leq N} C_{k-1}.$$

Second, we can eliminate the sum by multiplying both sides by N and subtracting the same formula for $N - 1$:

$$NC_N - (N - 1)C_{N-1} = N(N + 1) - (N - 1)N + 2C_{N-1}.$$

This formula simplifies to the recurrence

$$NC_N = (N + 1)C_{N-1} + 2N.$$

Third, dividing both sides by $N(N+1)$ gives a recurrence that telescopes:

$$\begin{aligned}\frac{C_N}{N+1} &= \frac{C_{N-1}}{N} + \frac{2}{N+1} \\ &= \frac{C_{N-2}}{N-1} + \frac{2}{N} + \frac{2}{N+1} \\ &= \vdots \\ &= \frac{C_2}{3} + \sum_{3 \leq k \leq N} \frac{2}{k+1}.\end{aligned}$$

This exact answer is nearly equal to a sum that is easily approximated by an integral (see Section 2.3):

$$\frac{C_N}{N+1} \approx 2 \sum_{1 \leq k < N} \frac{1}{k} \approx 2 \int_1^N \frac{1}{x} dx = 2 \ln N,$$

which implies the stated result. Note that $2N \ln N \approx 1.39N \lg N$, so the average number of comparisons is only about 39 percent higher than in the best case. ■

This analysis assumes that the file to be sorted comprises randomly ordered records with distinct keys, but the implementation in Programs 7.1 and 7.2 can run slowly in some cases when the keys are not necessarily distinct and not necessarily in random order, as illustrated in Figure 7.4. If the sort is to be used a great many times or if it is to be used to sort a huge file (or, in particular, if it is to be used as a general-purpose library sort that will be used to sort files of unknown characteristics), then we need to consider several of the improvements discussed in Sections 7.5 and 7.6 that can make it much less likely that a bad case will occur in practice, while also reducing the average running time by 20 percent.

Exercises

7.6 Give six files of 10 elements for which quicksort (Program 7.1) uses the same number of comparisons as the worst-case file (when all the elements are in order).

7.7 Write a program to compute the exact value of C_N , and compare the exact value with the approximation $2N \ln N$ for $N = 1000, 10000$, and 100000 .

○ **7.8** About how many comparisons will quicksort (Program 7.1) make when sorting a file of N equal elements?

- 7.9 About how many comparisons will quicksort (Program 7.1) make when sorting a file consisting of N items that have just two different key values (k items with one value, $N - k$ items with the other)?
- 7.10 Write a program that produces a best-case file for quicksort: a file of N distinct elements with the property that every partition will produce subfiles that differ in size by at most 1.

7.3 Stack Size

As we did in Chapter 3, we can use an explicit pushdown stack for quicksort, thinking of the stack as containing work to be done in the form of subfiles to be sorted. Any time that we need a subfile to process, we pop the stack. When we partition, we create two subfiles to be processed and push both on the stack. In the recursive implementation in Program 7.1, the stack maintained by the system holds this same information.

For a random file, the maximum size of the stack is proportional to $\log N$ (see *reference section*), but the stack can grow to size proportional to N for a degenerate case, as illustrated in Figure 7.5. Indeed, the very worst case is when the input file is already sorted. The potential for stack growth proportional to the size of the original file is a subtle but real difficulty with a recursive implementation of quicksort: There is always an underlying stack, and a degenerate case on a large file could cause the program to terminate abnormally because of lack of memory—behavior obviously undesirable for a library sorting routine. (Actually, we likely would run out of time before running out of space.) It is difficult to provide a *guarantee* against this behavior, but we shall see in Section 7.5 that it is not difficult to provide safeguards that make such degenerate cases extremely unlikely to occur.

Program 7.3 is a nonrecursive implementation that addresses this problem by checking the sizes of the two subfiles and putting the larger of the two on the stack first. Figure 7.6 illustrates this policy. Comparing this example with Figure 7.1, we see that the subfiles are not changed by this policy; only the order in which they are processed is changed. Thus, we save on space costs without affecting time costs.

The policy of putting the larger of the small subfiles on the stack ensures that each entry on the stack is no more than one-half of the size of the one below it, so that the stack needs to contain room for

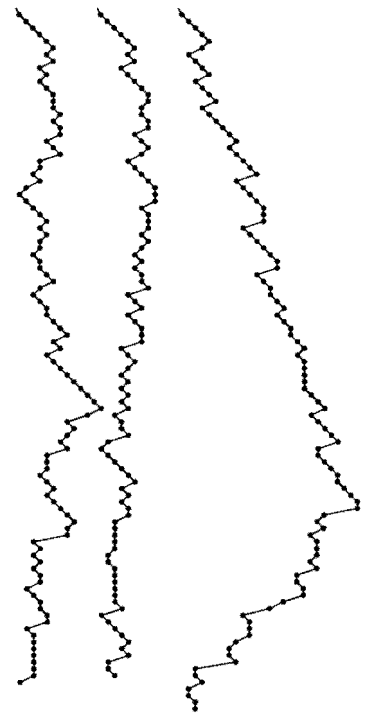


Figure 7.5
Stack size for quicksort

The recursive stack for quicksort does not grow large for random files, but can take excessive space for degenerate files. The stack sizes for two random files (left, center) and that for a partially ordered file (right) are plotted here.

This technique does not necessarily work in a truly recursive implementation, because it depends on *end- or tail-recursion removal*. If the last action of a procedure is to call another procedure, some programming environments will arrange things such that local variables are cleared from the stack *before*, rather than after, the call. Without end-recursion removal, we cannot guarantee that the stack size will be small for quicksort. For example, a call to quicksort for a file of size N that is already sorted will result in a recursive call to such a file of size $N - 1$, in turn resulting in a recursive call for such a file of size $N - 2$, and so on, ultimately resulting in a stack depth proportional to N . This observation would seem to suggest using a nonrecursive implementation to guard against excessive stack growth. On the other hand, some C compilers automatically remove end recursion, and many machines have direct hardware support for function calls—the nonrecursive implementation in Program 7.3 might therefore actually be slower than the recursive implementation in Program 7.1 in such environments.

Figure 7.7 further illustrates the point that the nonrecursive method processes the same subfiles (in a different order) as does the recursive method for any file. It shows a tree structure with the partitioning element at the root and the trees corresponding to the left and right subfiles as left and right children, respectively. Using the recursive implementation of quicksort corresponds to visiting the nodes of this tree in preorder; the nonrecursive implementation corresponds to a visit-the-smaller-subtree-first traversal rule.

When we use an explicit stack, as we did in Program 7.3, we avoid some of the overhead implicit in a recursive implementation, although modern programming systems do not incur much overhead for such simple programs. Program 7.3 can be further improved. For example, it puts both subfiles on the stack, only to have the top one immediately popped off; we could change it to set the variables l and r directly. Also, the test for $r \leq 1$ is done as subfiles come off the stack, whereas it would be more efficient never to put such subfiles on the stack (see Exercise 7.14). This case might seem insignificant, but the recursive nature of quicksort actually ensures that a large fraction of the subfiles during the course of the sort are of size 0 or 1. Next, we examine an important improvement to quicksort that gains efficiency by expanding upon this idea, handling all small subfiles in as efficient a manner as possible.

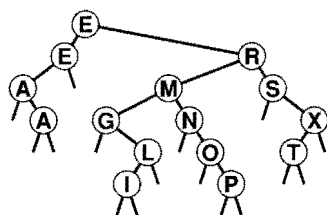


Figure 7.7
Quicksort partitioning tree

If we collapse the partitioning diagrams in Figures 7.1 and 7.6 by connecting each partitioning element to the partitioning element used in its two subfiles, we get this static representation of the partitioning process (in both cases). In this binary tree, each subfile is represented by its partitioning element (or by itself, if it is of size 1), and the subtrees of each node are the trees representing the subfiles after partitioning. For clarity, null subfiles are not shown here, although our recursive versions of the algorithm do make recursive calls with $r < 1$ when the partitioning element is the smallest or largest element in the file. The tree itself does not depend on the order in which the subfiles are partitioned. Our recursive implementation of quicksort corresponds to visiting the nodes of this tree in preorder; our nonrecursive implementation corresponds to a visit-the-smaller-subtree-first rule.

Exercises

- ▷ 7.11 Give, in the style of Figure 5.5, the stack contents after each pair of *push* and *pop* operations, when Program 7.3 is used to sort a file with the keys EASYQUESTION.
- ▷ 7.12 Answer Exercise 7.11 for the case where we always push the right subfile, then the left subfile (as is the case in the recursive implementation).
- 7.13 Complete the proof of Property 7.3, by induction.
- 7.14 Revise Program 7.3 such that it never puts on the stack subfiles with $r \leq 1$.
- ▷ 7.15 Give the maximum stack size required by Program 7.3 when $N = 2^n$.
- 7.16 Give the maximum stack sizes required by Program 7.3 when $N = 2^n - 1$ and $N = 2^n + 1$.
- 7.17 Would it be reasonable to use a queue instead of a stack for a nonrecursive implementation of quicksort? Explain your answer.
- 7.18 Determine and report whether your programming environment implements end-recursion removal.
- 7.19 Run empirical studies to determine the average stack size used by the basic recursive quicksort algorithm for random files of N elements, for $N = 10^3, 10^4, 10^5$, and 10^6 .
- 7.20 Find the average number of subfiles of size 0, 1, and 2 when quicksort is used to sort a random file of N elements.

7.4 Small Subfiles

A definite improvement to quicksort arises from the observation that a recursive program is guaranteed to call itself for many small subfiles, so it should use as good a method as possible when it encounters small subfiles. One obvious way to arrange for it to do so is to change the test at the beginning of the recursive routine from a `return` to a call on insertion sort, as follows:

```
if (r-l ≤ M) insertion(a, l, r);
```

Here, M is some parameter whose exact value depends upon the implementation. We can determine the best value for M either through analysis or with empirical studies. It is typical to find in such studies that the running time does not vary much for M in the range from about 5 to about 25, with the running time for M in this range on

the order of 10 percent less than for the naive choice $M = 1$ (see Figure 7.8).

A slightly easier way to handle small subfiles, which is also slightly more efficient than insertion sorting them as they are encountered, is just to change the test at the beginning to

```
if (r-l <= M) return;
```

That is, we simply ignore small subfiles during partitioning. In a nonrecursive implementation, we could do so by not putting any files of size less than M on the stack, or, alternatively, by ignoring all files of size less than M that are found on the stack. After partitioning, what is left is a file that is almost sorted. As discussed in Section 6.5, however, insertion sort is the method of choice for such files. That is, insertion sort will work about as well for such a file as for the collection of little files that it would get if it were being used directly. This method should be used with caution, because insertion sort is likely to work even if quicksort has a bug that causes it not to sort at all. Excessive cost may be the only sign that something went wrong.

Figure 7.9 illustrates this process for a larger file. Even with a relatively large cutoff for small subfiles, the quicksort part of the process runs quickly because relatively few elements are involved in partitioning steps. The insertion sort that finishes the job also runs quickly because it starts with a file that is nearly in order.

This technique can be used to good advantage whenever we are dealing with a recursive algorithm. Because of their very nature, we can be sure that *all* recursive algorithms will be processing small problem instances for a high percentage of the time; we generally do have available a low-overhead brute-force algorithm for small cases; and we therefore generally can improve overall timings with a hybrid algorithm.

Exercises

- 7.21 Are sentinel keys needed if insertion sort is called directly from within quicksort?
- 7.22 Instrument Program 7.1 to give the percentage of the comparisons used in partitioning files of size less than 10, 100, and 1000, and print out the percentages when you sort random files of N elements, for $N = 10^3, 10^4, 10^5$, and 10^6 .
- 7.23 Implement a recursive quicksort with a cutoff to insertion sort for subfiles with less than M elements, and empirically determine the value of M

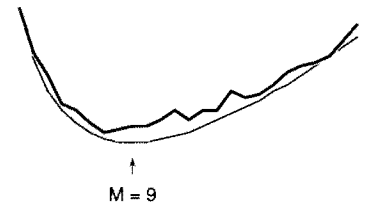


Figure 7.8
Cutoff for small subfiles

Choosing the optimal value for the cutoff for small subfiles results in about a 10 percent improvement in the average running time. Choosing the value precisely is not critical; values from a broad range (from about 5 to about 20) will work about as well for most implementations. The thick line (top) was obtained empirically; the thin line (bottom) was derived analytically.

Figure 7.9
Comparisons in quicksort

Quicksort subfiles are processed independently. This picture shows the result of partitioning each subfile during a sort of 200 elements with a cutoff for files of size 15 or less. We can get a rough idea of the total number of comparisons by counting the number of marked elements by column vertically. In this case, each array position is involved in only six or seven subfiles during the sort.



for which Program 7.4 runs fastest in your computing environment to sort random files of N elements, for $N = 10^3, 10^4, 10^5$, and 10^6 .

7.24 Solve Exercise 7.23 using a nonrecursive implementation.

7.25 Solve Exercise 7.23, for the case when the records to be sorted contain a key and b pointers to other information (but we are not using a pointer sort).

- 7.26 Write a program that plots a histogram (see Program 3.7) of the subfile sizes left for insertion sort when you run quicksort for a file of size N with a cutoff for subfiles of size less than M . Run your program for $M = 10, 100$, and 1000 and $N = 10^3, 10^4, 10^5$, and 10^6 .

7.27 Run empirical studies to determine the average stack size used by quicksort with cutoff for files of size M , when sorting random files of N elements, for $M = 10, 100$, and 1000 and $N = 10^3, 10^4, 10^5$, and 10^6 .

7.5 Median-of-Three Partitioning

Another improvement to quicksort is to use a partitioning element that is more likely to divide the file near the middle. There are several possibilities here. A safe choice to avoid the worst case is to use a random element from the array for a partitioning element. Then, the worst case will happen with negligibly small probability. This method is a simple example of a *probabilistic algorithm*—one that uses randomness to achieve good performance with high probability, regardless of the arrangement of the input. We will see numerous examples later in the book of the utility of randomness in algorithm design, particularly when bias in the input is suspected. For quicksort, it may be overkill in practice to put in a full random-number generator just for this purpose: simple arbitrary choices can also be effective.

Another well-known way to find a better partitioning element is to take a sample of three elements from the file, then to use the median of the three for the partitioning element. By choosing the three elements from the left, middle, and right of the array, we can incorporate sentinels into this scheme as well: sort the three elements (using the three-exchange method in Chapter 6), then exchange the one in the middle with a $[r-1]$, and then run the partitioning algorithm on a $[l+1], \dots, a[r-2]$. This improvement is called the *median-of-three* method.

The median-of-three method helps quicksort in three ways. First, it makes the worst case much more unlikely to occur in any actual

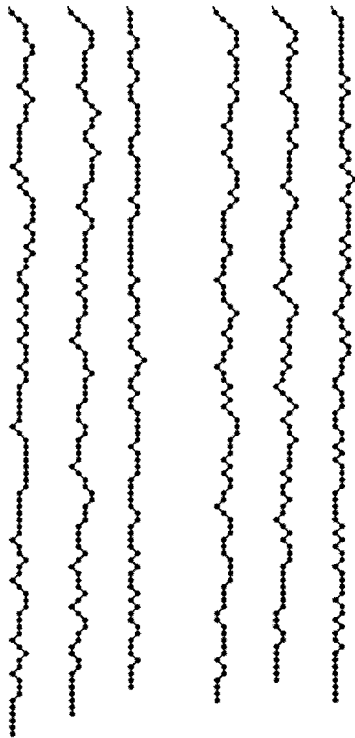


Figure 7.10
Stack size for improved versions of quicksort

Sorting the smaller subfile first guarantees that the stack size will be logarithmic at worst. Plotted here are the stack sizes for the same files as in Figure 7.5, with the smaller of the subfiles sorted first during the sort (left) and with the median-of-three modification added (right). These diagrams are not indicative of running time; that variable depends on the size of the files on the stack, rather than only their number. For example, the third file (partially sorted) does not require much stack space, but leads to a slow sort because the subfiles being processed are usually large.

sort. For the sort to take N^2 time, two out of the three elements examined must be among the largest or among the smallest elements in the file, and this event must happen consistently through most of the partitions. Second, it eliminates the need for a sentinel key for partitioning, because this function is served by one of the three elements that are examined before partitioning. Third, it reduces the total average running time of the algorithm by about 5 percent.

The combination of using the median-of-three method with a cutoff for small subfiles can improve the running time of quicksort over the naive recursive implementation by 20 to 25 percent. Program 7.4 is an implementation that incorporates all these improvements.

We might consider continuing to improve the program by removing recursion, replacing the subroutine calls by inline code, using sentinels, and so forth. However, on modern machines, such procedure calls are normally efficient, and they are not in the inner loop. More important, the use of the cutoff for small subfiles tends to compensate for any extra overhead that may be involved (outside the inner loop). The primary reason to use a nonrecursive implementation with an explicit stack is to be able to provide guarantees on limiting the stack size (see Figure 7.10).

Further algorithmic improvements are possible (for example, we could use the median of five or more elements), but the amount of time gained will be marginal for random files. We *can* realize significant time savings by coding the inner loops (or the whole program) in assembly or machine language. These observations have been validated on numerous occasions by experts with serious sorting applications (see *reference section*).

For randomly ordered files, the first exchange in Program 7.4 is superfluous. We include it not just because it leads to optimal partitioning for files already in order, but also because it protects against anomalous situations that might occur in practice (see, for example, Exercise 7.33). Figure 7.11 illustrates the effectiveness of involving the middle element in the partitioning decision, for various types of files.

The median-of-three method is a special case of the general idea that we can sample an unknown file and use properties of the sample to estimate properties of the whole file. For quicksort, we want to estimate the median to balance the partitioning. It is the nature of the algorithm that we do not need a particularly good estimate (and may

Program 7.4 Improved quicksort

Choosing the median of the first, middle, and final elements as the partitioning element and cutting off the recursion for small subfiles can significantly improve the performance of quicksort. This implementation partitions on the median of the first, middle, and final elements in the array (otherwise leaving these elements out of the partitioning process). Files of size 11 or smaller are ignored during partitioning; then, insertion from Chapter 6 is used to finish the sort.

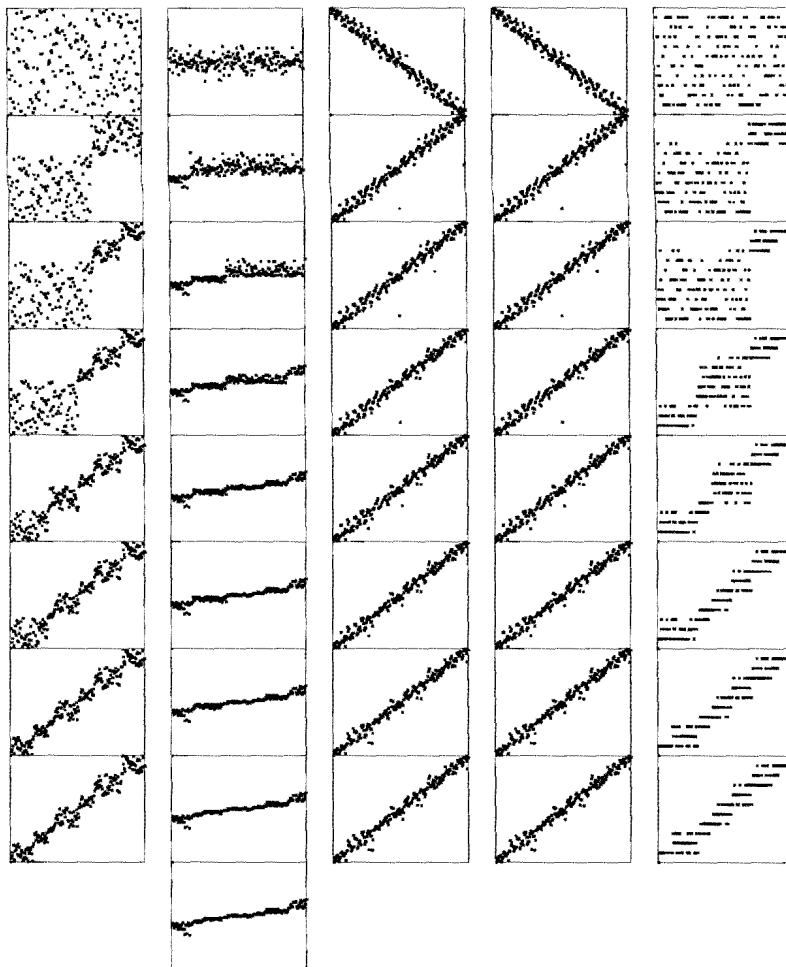
```
#define M 10
void quicksort(Item a[], int l, int r)
{
    int i;
    if (r-l <= M) return;
    exch(a[(l+r)/2], a[r-1]);
    compexch(a[l], a[r-1]);
    compexch(a[l], a[r]);
    compexch(a[r-1], a[r]);
    i = partition(a, l+1, r-1);
    quicksort(a, l, i-1);
    quicksort(a, i+1, r);
}
void sort(Item a[], int l, int r)
{
    quicksort(a, l, r);
    insertion(a, l, r);
}
```

not want one if such an estimate is expensive to compute); we just want to avoid a particularly bad estimate. If we use a random sample of just one element, we get a randomized algorithm that is virtually certain to run quickly, no matter what the input. If we randomly choose three or five elements from the file, then use the median of that sample for partitioning, we get a better partition, but the improvement is offset by the cost of taking the sample.

Quicksort is widely used because it runs well in a variety of situations. Other methods might be more appropriate for particular cases that might arise, but quicksort handles more types of sorting problems than are handled by many other methods, and it is often significantly

Figure 7.11
Dynamic characteristics of
median-of-three quicksort
on various types of files

The median-of-three modification (particularly, using the middle element of the file) does a good job of making the partitioning process more robust. The degenerate types of files shown in Figure 7.4 are handled particularly well. Another option that achieves this same goal is to use a random partitioning element.



faster than alternative approaches. Table 7.1 gives empirical results in support of some of these comments.

Exercises

7.28 Our implementation of the median-of-three method is careful to ensure that the sampled elements do not participate in the partitioning process. One reason is that they can serve as sentinels. Give another reason.

7.29 Implement a quicksort based on partitioning on the median of a random sample of five elements from the file. Make sure that the elements of the sample do not participate in partitioning (see Exercise 7.28). Compare the

Table 7.1 Empirical study of basic quicksort algorithms

Quicksort (Program 7.1) is more than twice as fast as shellsort (Program 6.6) for large randomly ordered files. A cutoff for small subfiles and the median-of-three improvement (Program 7.4) lower the running time by about 10 percent each.

N	shellsort	Basic quicksort			Median-of-three quicksort		
		$M = 0$	$M = 10$	$M = 20$	$M = 0$	$M = 10$	$M = 20$
12500	6	2	2	2	3	2	3
25000	10	5	5	5	5	4	6
50000	26	11	10	10	12	9	14
100000	58	24	22	22	25	20	28
200000	126	53	48	50	52	44	54
400000	278	116	105	110	114	97	118
800000	616	255	231	241	252	213	258

performance of your algorithm with the median-of-three method for large random files.

7.30 Run your program from Exercise 7.29 on large nonrandom files—for example, sorted files, files in reverse order, or files with all keys equal. How does its performance for these files differ from its performance for random files?

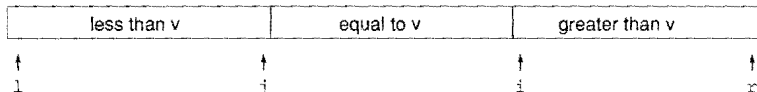
- **7.31** Implement a quicksort based on using a sample of size $2^k - 1$. First, sort the sample, then, arrange to have the recursive routine partition on the median of the sample and to move the two halves of the rest of the sample to each subfile, such that they can be used in the subfiles, without having to be sorted again. This algorithm, which uses about $N \lg N$ comparisons when k is about $\lg N - \lg \lg N$, is called *samplesort*.
- **7.32** Run empirical studies to determine the best value of the sample size in samplesort (see Exercise 7.31), for $N = 10^3$, 10^4 , 10^5 , and 10^6 . Does it matter whether quicksort or samplesort is used to sort the sample?
- **7.33** Show that Program 7.4, if changed to omit the first exchange and to scan over keys equal to the partitioning element, runs in quadratic time on a file that is in reverse order.

7.6 Duplicate Keys

Files with large numbers of duplicate sort keys arise frequently in applications. For example, we might wish to sort a large personnel file by year of birth, or even to use a sort to separate females from males.

When there are many duplicate keys present in the file to be sorted, the quicksort implementations that we have considered do not have unacceptably poor performance, but they can be substantially improved. For example, a file that consists solely of keys that are equal (just one value) does not need to be processed further, but our implementations so far keep partitioning down to small subfiles, no matter how big the file is (see Exercise 7.8). In a situation where there are large numbers of duplicate keys in the input file, the recursive nature of quicksort ensures that subfiles consisting solely of items with a single key value will occur often, so there is potential for significant improvement.

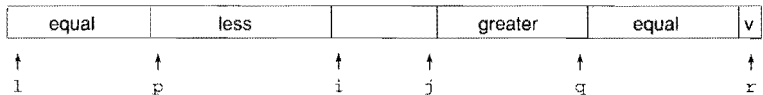
One straightforward idea is to partition the file into *three* parts, one each for keys smaller than, equal to, and larger than the partitioning element:



Accomplishing this partitioning is more complicated than the two-way partitioning that we have been using, and various different methods have been suggested for the task. It was a classical programming exercise popularized by Dijkstra as the *Dutch National Flag problem*, because the three possible key categories might correspond to the three colors on the flag (see reference section). For quicksort, we add the constraint that a single pass through the file must do the job—an algorithm that involves two passes through the data would slow down quicksort by a factor of two, even if there are no duplicate keys at all.

A clever method invented by Bentley and McIlroy in 1993 for three-way partitioning works by modifying the standard partitioning scheme as follows: Keep keys equal to the partitioning element that are encountered in the left subfile at the left end of the file, and keep keys equal to the partitioning element that are encountered in the right

subfile at the right end of the file. During the partitioning process, we maintain the following situation:



Then, when the pointers cross and the precise location for the equal keys is known, we swap into position all the items with keys equal to the partitioning element. This scheme does not quite meet the requirement that three-way partitioning be accomplished in one pass through the file, but the extra overhead for duplicate keys is proportional to only the number of duplicate keys found. This fact has two implications: First, the method works well even if there are no duplicate keys, since there is no extra overhead. Second, the method is linear time when there is only a constant number of key values: Each partitioning phase removes from the sort all the keys with the same value as the partitioning element, so each key can be involved in at most a constant number of partitions.

Figure 7.12 illustrates the three-way partitioning algorithm on a sample file, and Program 7.5 is a quicksort implementation based on the method. The implementation requires the addition of just two if statements in the exchange loop, and just two for loops to complete partitioning by putting the keys equal to the partitioning element into position. It seems to require less code than other alternatives for maintaining three partitions. More important, it not only handles duplicate keys in as efficient a manner as possible, but also incurs a minimal amount of extra overhead in the case that there are no duplicate keys.

Exercises

- ▷ 7.34 Explain what happens when Program 7.5 is run on a randomly ordered file with (i) two distinct key values, and (ii) three distinct key values.
- 7.35 Modify Program 7.1 to return if all keys in the subfile are equal. Compare the performance of your program to Program 7.1 for large random files with keys having t distinct values for $t = 2, 5$, and 10.
- 7.36 Suppose that we scan over keys equal to the partitioning element in Program 7.2 instead of stopping the scans when we encounter them. Show that the running time of Program 7.1 would be quadratic in this case.

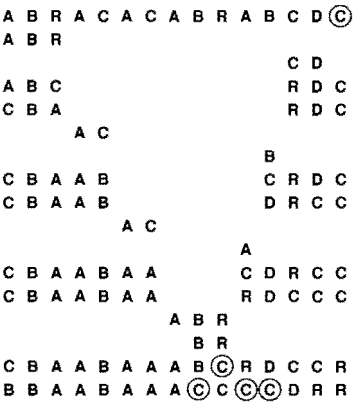


Figure 7.12
Three-way partitioning

This diagram depicts the process of putting all keys equal to the partitioning element into position. As in Figure 7.2, we scan from the left to find an element that is not smaller than the partitioning element and from the right to find an element that is not larger than the partitioning element, then exchange them. If the element on the left after the exchange is equal to the partitioning element, we exchange it to the left end of the array; we proceed similarly on the right. When the pointers cross, we put the partitioning element into position as before (next-to-bottom line), then exchange all the keys equal to it into position on either side of it (bottom line).

Program 7.5 Quicksort with three-way partitioning

This program is based on partitioning the array into three parts: elements smaller than the partitioning element (in $a[l], \dots, a[j]$); elements equal to the partitioning element (in $a[j+1], \dots, a[i-1]$); and elements larger than the partitioning element (in $a[i], \dots, a[r]$). Then the sort can be completed with two recursive calls, one for the smaller keys and one for the larger keys.

To accomplish the objective, the program keeps keys equal to the partitioning element on the left between l and ll and on the right between rr and r . In the partitioning loop, after the scan pointers stop and the items at i and j are exchanged, it checks each of those items to see whether it is equal to the partitioning element. If the one now on the left is equal to the partitioning element, it is exchanged into the left part of the array; if one now on the right is equal to the partitioning element, it is exchanged into the right part of the array.

After the pointers cross, the two ends of the array with elements equal to the partitioning element are exchanged back to the middle. Then those keys are in position and can be excluded from the subfiles for the recursive calls.

```
#define eq(A, B) (!less(A, B) && !less(B, A))
void quicksort(Item a[], int l, int r)
{ int i, j, k, p, q; Item v;
  if (r <= l) return;
  v = a[r]; i = l-1; j = r; p = l-1; q = r;
  for (;;)
  {
    while (less(a[++i], v)) ;
    while (less(v, a[--j])) if (j == l) break;
    if (i >= j) break;
    exch(a[i], a[j]);
    if (eq(a[i], v)) { p++; exch(a[p], a[i]); }
    if (eq(v, a[j])) { q--; exch(a[q], a[j]); }
  }
  exch(a[i], a[r]); j = i-1; i = i+1;
  for (k = l ; k < p; k++, j--) exch(a[k], a[j]);
  for (k = r-1; k > q; k--, i++) exch(a[k], a[i]);
  quicksort(a, l, j);
  quicksort(a, i, r);
}
```

- 7.37 Prove that the running time of the program in Exercise 7.36 is quadratic for all files with $O(1)$ distinct key values.

7.38 Write a program to determine the number of distinct keys that occur in a file. Use your program to count the distinct keys in random files of N integers in the range 0 to $M - 1$, for $M = 10, 100$, and 1000 , and for $N = 10^3, 10^4, 10^5$, and 10^6 .

7.7 Strings and Vectors

When the sort keys are strings, we could use an abstract-string type implementation like Program 6.11 with the quicksort implementations in this chapter. Although this approach provides a correct and efficient implementation (faster than any other method we have seen so far, for large files), there is a hidden cost that is interesting to consider.

The problem lies in the cost of the `strcmp` function, which always compares two strings by proceeding from left to right, comparing strings character by character, taking time proportional to the number of leading characters that match in the two strings. For the later partitioning stages of quicksort, when keys are close together, this match might be relatively long. As usual, because of the recursive nature of quicksort, nearly all the cost of the algorithm is incurred in the later stages, so examining improvements there is worthwhile.

For example, consider a subfile of size 5 containing the keys `discreet`, `discredit`, `discrete`, `discrepancy`, and `discretion`. All the comparisons used for sorting these keys examine at least seven characters, when it would suffice to start at the seventh character, if the extra information that the first six characters are equal were available.

The three-way partitioning procedure that we considered in Section 7.6 provides an elegant way to take advantage of this observation. At each partitioning stage, we examine just one character (say the one at position d), assuming that the keys to be sorted are equal in positions 0 through $d-1$. We do a three-way partition with keys whose d th character is smaller than the d th character of the partitioning element on the left, those whose d th character is equal to the d th character of the partitioning element in the middle, and those whose d th character is larger than the d th character of the partitioning element on the right. Then, we proceed as usual, *except* that we sort the middle sub-

Table 7.2 Empirical study of quicksort variants

This table gives relative costs for several different versions of quicksort on the task of sorting the first N words of *Moby Dick*. Using insertion sort directly for small subfiles, or ignoring them and insertion sorting the same file afterward, are equally effective strategies, but the cost savings is slightly less than for integer keys (see Table 7.1) because comparisons are more expensive for strings. If we do not stop on duplicate keys when partitioning, then the time to sort a file with all keys equal is quadratic; the effect of this inefficiency is noticeable on this example, because there are numerous words that appear with high frequency in the data. For the same reason, three-way partitioning is effective; it is 30 to 35 percent faster than the system sort.

N	V	I	M	Q	X	T
12500	8	7	6	10	7	6
25000	16	14	13	20	17	12
50000	37	31	31	45	41	29
100000	91	78	76	103	113	68

Key:

- V Quicksort (Program 7.1)
- I Insertion sort for small subfiles
- M Ignore small subfiles, insertion sort afterward
- Q System `qsort`
- X Scan over duplicate keys (goes quadratic when keys all equal)
- T Three-way partitioning (Program 7.5)

file, starting at character $d+1$. It is not difficult to see that this method leads to a proper sort on strings, which turns out to be very efficient (see Table 7.2). We have here a convincing example of the power of thinking (and programming) recursively.

To implement the sort, we need a more general abstract type that allows access to characters of keys. The way in which strings are handled in C makes the implementation of this method particularly straightforward. However, we defer considering the implementation in detail until Chapter 10, where we consider a variety of techniques for sorting that take advantage of the fact that sort keys can often be easily decomposed into smaller pieces.

This approach generalizes to handle multidimensional sorts, where the sort keys are vectors and the records are to be rearranged such that the first components of the keys are in order, then those with first component equal are in order by second component, and so forth. If the components do not have duplicate keys, the problem reduces to sorting on the first component; in a typical application, however, each of the components may have only a few distinct values, and three-way partitioning (moving to the next component for the middle partition) is appropriate. This case was discussed by Hoare in his original paper, and is an important application.

Exercises

- 7.39 Discuss the possibility of improving selection, insertion, bubble, and shell sorts for strings.
- 7.40 How many characters are examined by the standard quicksort algorithm (Program 7.1, using the string type in Program 6.11) when sorting a file consisting of N strings of length t , all of which are equal? Answer the same question for the modification proposed in the text.

7.8 Selection

An important application related to sorting but for which a full sort is not required is the operation of finding the median of a set of numbers. This operation is a common computation in statistics and in various other data-processing applications. One way to proceed would be to sort the numbers and to look at the middle one, but we can do better, using the quicksort partitioning process.

The operation of finding the median is a special case of the operation of *selection*: finding the k th smallest of a set of numbers. Because an algorithm cannot guarantee that a particular item is the k th smallest without having examined and identified the $k - 1$ elements that are smaller and the $N - k$ elements that are larger, most selection algorithms can return all the k smallest elements of a file without a great deal of extra calculation.

Selection has many applications in the processing of experimental and other data. The use of the median and other *order statistics* to divide a file into smaller groups is common. Often, only a small part of a large file is to be saved for further processing; in such cases, a program that can select, say, the top 10 percent of the elements of the

Program 7.6 Selection

This procedure partitions an array about the $(k-1)$ th smallest element (the one in $a[k]$): It rearranges the array to leave $a[1], \dots, a[k-1]$ less than or equal to $a[k]$, and $a[k+1], \dots, a[r]$ greater than or equal to $a[k]$.

For example, we could call `select(a, 0, N-1, N/2)` to partition the array on the median value, leaving the median in $a[N/2]$.

```
select(Item a[], int l, int r, int k)
{
    int i;
    if (r <= l) return;
    i = partition(a, l, r);
    if (i > k) select(a, l, i-1, k);
    if (i < k) select(a, i+1, r, k);
}
```

A S O R T I N G E X A M P L E
A A E (E) T I N G O X S M P L R
L I N G O P M (R) X T S
L I G (M) O P N
A A E E L I G (M) O P N R X T S

Figure 7.13
Selection of the median

For the keys in our sorting example, partitioning-based selection uses only three recursive calls to find the median. On the first call, we seek the eighth smallest in a file of size 15, and partitioning gives the fourth smallest (the E); so on the second call, we seek the fourth smallest in a file of size 11, and partitioning gives the eighth smallest (the R); so on the third call, we seek the fourth smallest in a file of size 7, and find it (the M). The file is rearranged such that the median is in place, with smaller elements to the left and larger elements to the right (equal elements could be on either side), but it is not fully sorted.

file might be more appropriate than a full sort. Another important example is the use of partitioning about the median as a first step in many divide-and-conquer algorithms.

We have already seen an algorithm that we can adapt directly to selection. If k is extremely small, then selection sort (see Chapter 6) will work well, requiring time proportional to Nk : first find the smallest element, then find the second smallest by finding the smallest of the remaining items, and so forth. For slightly larger k , we shall see methods in Chapter 9 that we could adapt to run in time proportional to $N \log k$.

A selection method that runs in linear time on the average for all values of k follows directly from the partitioning procedure used in quicksort. Recall that quicksort's partitioning method rearranges an array $a[1], \dots, a[r]$ and returns an integer i such that $a[1]$ through $a[i-1]$ are less than or equal to $a[i]$, and $a[i+1]$ through $a[r]$ are greater than or equal to $a[i]$. If k is equal to i , then we are done. Otherwise, if $k < i$, then we need to continue working in the left subfile; if $k > i$, then we need to continue working in the right subfile. This approach leads immediately to the recursive program for selection that is Program 7.6. An example of this procedure in operation on a small file is given in Figure 7.13.

Program 7.7 Nonrecursive selection

A nonrecursive implementation of selection simply does a partition, then moves the left pointer in if the partition fell to the left of the position sought, or moves the right pointer in if the partition fell to the right of the position sought.

```
select(Item a[], int l, int r, int k)
{
    while (r > l)
    { int i = partition(a, l, r);
      if (i >= k) r = i-1;
      if (i <= k) l = i+1;
    }
}
```

Program 7.7 is a nonrecursive version that follows directly from the recursive version in Program 7.6. Since that program always ends with a single call on itself, we simply reset the parameters and go back to the beginning. That is, we remove the recursion without needing a stack, also eliminating the calculations involving k by keeping k as an array index.

Property 7.4 *Quicksort-based selection is linear time on the average.*

As we did for quicksort, we can argue (roughly) that, on an extremely large file, each partition should roughly split the array in half, so the whole process should require about $N + N/2 + N/4 + N/8 + \dots = 2N$ comparisons. And, as it was for quicksort, this rough argument is not far from the truth. An analysis similar to, but significantly more complex than, that given in Section 7.2 for quicksort (*see reference section*) leads to the result that the average number of comparisons is about

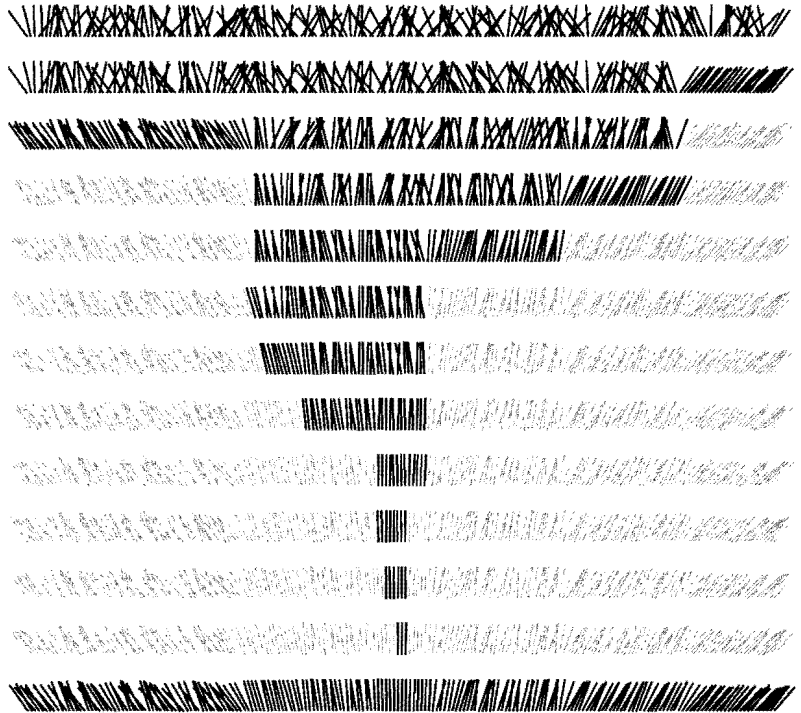
$$2N + 2k \ln(N/k) + 2(N - k) \ln(N/(N - k)),$$

which is linear for any allowed value of k . For $k = N/2$, this formula evaluates to give the result that about $(2 + 2 \ln 2)N$ comparisons are required to find the median. ■

An example showing how this method finds the median in a large file is depicted in Figure 7.14. There is only one subfile, which is cut

Figure 7.14
Selection of the median by
partitioning

The selection process involves partitioning the subfile that contains the element sought, moving the left pointer to the right or the right pointer to the left depending on where the partition falls.



down in size by a constant factor on each call, so the procedure finishes in $O(\log N)$ steps. We can speed up the program with sampling, but we need to exercise care in doing so (see Exercise 7.45).

The worst case is about the same as for quicksort—using this method to find the smallest element in a file that is already in sorted order would result in a quadratic running time. It is possible to modify this quicksort-based selection procedure such that its running time is *guaranteed* to be linear. These modifications, although theoretically important, are extremely complex and are not at all practical.

Exercises

- 7.41** About how many comparisons are required, on the average, to find the smallest of N elements using `select`?
- 7.42** About how many comparisons are required, on the average, to find the αN th smallest element using `select`, for $\alpha = 0.1, 0.2, \dots, 0.9$?
- 7.43** How many comparisons are required in the worst case to find the median of N elements using `select`?

- 7.44 Write an efficient program to rearrange a file such that all the elements with keys equal to the median are in place, with smaller elements to the left and larger elements to the right.
- 7.45 Investigate the idea of using sampling to improve selection. *Hint:* Using the median may not always be helpful.
 - 7.46 Implement a selection algorithm based on three-way partitioning for large random files with keys having t distinct values for $t = 2, 5$, and 10 .



Merging and Mergesort

THE QUICKSORT FAMILY of algorithms that we studied in Chapter 7 are based on the *selection* operation: finding the k th smallest element in a file. We saw that performing selection is akin to dividing a file into two parts, the k smallest elements and the $N - k$ largest elements. In this chapter, we examine a family of sorting algorithms based on a complementary process, *merging*: combining two ordered files to make one larger ordered file. Merging is the basis for a straightforward divide-and-conquer (see Section 5.2) sorting algorithm, and for a bottom-up counterpart, both of which are easy to implement.

Selection and merging are complementary operations in the sense that selection splits a file into two independent files, whereas merging joins two independent files to make one file. The contrast between these operations also becomes evident when we apply the divide-and-conquer paradigm to create a sorting method. We can rearrange the file such that, when two parts are sorted, the whole file is ordered; alternatively, we can break the file into two parts to be sorted, and then combine the ordered parts to make the whole ordered file. We have already seen what happens in the first instance: that is quicksort, which consists of a selection procedure followed by two recursive calls. In this chapter, we shall look at *mergesort*, which is quicksort's complement in that it consists of two recursive calls followed by a merging procedure.

One of mergesort's most attractive properties is that it sorts a file of N elements in time proportional to $N \log N$, no matter what the input. In Chapter 9, we shall see another algorithm that is guaranteed

to finish in time proportional to $N \log N$; it is called *heapsort*. The prime disadvantage of mergesort is that extra space proportional to N is needed in straightforward implementations. We can overcome this handicap, but doing so is sufficiently complicated and costly that it is generally not worthwhile in practice, particularly in light of the heapsort alternative. Mergesort is no more difficult to code than is heapsort, and the length of the inner loop is between those of quicksort and heapsort, so mergesort is worth considering if speed is of the essence, bad worst-case performance cannot be tolerated, and extra space is available.

A guaranteed $N \log N$ running time can be a liability. For example, in Chapter 6, we saw that there are methods that can adapt to run in linear time in certain special situations, such as when there is a significant amount of order in the file, or when there are only a few distinct keys. By contrast, the running time of mergesort depends primarily on only the number of input keys, and is virtually insensitive to their order.

Mergesort is a stable sort, and this feature tips the balance in its favor for applications where stability is important. Competitive methods such as quicksort and heapsort are not stable. Various techniques to make such methods stable tend to require extra space; mergesort's extra-space requirement thus becomes less significant if stability is a prime consideration.

Another feature of mergesort that is important in certain situations is that mergesort is normally implemented such that it accesses the data primarily sequentially (one item after the other). For example, mergesort is the method of choice for sorting a linked list, where sequential access is the only kind of access available. For similar reasons, as we shall see in Chapter 11, merging is often chosen as the basis for sorting on special-purpose and high-performance machines, because it is often the case that sequential access to data is fastest in such environments.

8.1 Two-Way Merging

Given two ordered input files, we can combine them into one ordered output file simply by keeping track of the smallest element in each file and entering a loop where the smaller of the two elements that are

Program 8.1 Merging

To combine two ordered arrays *a* and *b* into an ordered array *c*, we use a *for* loop that puts an element into *c* at each iteration. If *a* is exhausted, the element comes from *b*; if *b* is exhausted, the element comes from *a*; and if items remain in both, the smallest of the remaining elements in *a* and *b* goes to *c*. Beyond the implicit assumption that the arrays are ordered, this implementation assumes that the array *c* is disjoint from (that is, does not overlap or share storage with) *a* and *b*.

```
mergeAB(Item c[], Item a[], int N, Item b[], int M)
{ int i, j, k;
  for (i = 0, j = 0, k = 0; k < N+M; k++)
  {
    if (i == N) { c[k] = b[j++]; continue; }
    if (j == M) { c[k] = a[i++]; continue; }
    c[k] = (less(a[i], b[j])) ? a[i++] : b[j++];
  }
}
```

smallest in their files is moved to the output, continuing until both input files are exhausted. We shall look at several implementations of this basic abstract operation in this and the next section. The running time is linear in the number of elements in the output, as long as we can perform the operation of finding the next smallest element in a file in constant time, which is certainly the case for files that are in sorted order and represented with a data structure that supports constant-time sequential access, such as an array or a linked list. This procedure is *two-way merging*; in Chapter 11, we shall look in detail at *multiway merging*, when more than two files are involved. The most important application of multiway merging is external sorting, which is discussed in detail in that chapter.

To begin, let us suppose that we have two disjoint ordered arrays *a*[0], ..., *a*[*N*-1] and *b*[0], ..., *b*[*M*-1] of integers that we wish to merge into a third array *c*[0], ..., *c*[*N*+*M*-1]. The obvious strategy, which is easily implemented, is to choose successively for *c* the smallest remaining element from *a* and *b*, as shown in Program 8.1. This implementation is simple, but it has two important characteristics that we shall now examine.

First, the implementation assumes that the arrays are disjoint. In particular, if a and b are huge arrays, then a third (also huge) array c is needed to hold the output. Instead of using extra space proportional to the size of the merged file, it would be desirable to have an in-place method, so that, for example, we could combine the ordered files $a[1], \dots, a[m]$ and $a[m+1], \dots, a[r]$ into a single ordered file by moving the elements around within $a[1], \dots, a[r]$, without using a significant amount of other extra space. It is a worthwhile exercise to pause momentarily to consider how we might do that. This problem seems to be one that must be simple to solve; actually, however, the solutions that are known are complicated, especially by comparison to Program 8.1. Indeed, it is not easy to develop an algorithm for in-place merging that can outperform the alternative of using an in-place *sort*. We shall return to this issue in Section 8.2.

Merging has specific applications in its own right. For example, in a typical data-processing environment, we might need to maintain a large (ordered) data file, to which we will need to regularly add new entries. One approach is to *batch* each group of new entries—append them to the (much larger) main file, then resort the whole file. This situation is tailor-made for merging: A much more efficient strategy is to sort the (small) batch of new entries, then to merge the resulting small file with the large main file. Merging has many other similar applications that make its study worthwhile. Our prime interest in this chapter will be the sorting methods that are based on merging.

Exercises

8.1 Suppose that an ordered file of size N is to be combined with an unordered file of size M , with M much smaller than N . How many times faster than resorting is the suggested merge-based method, as a function of M , for $N = 10^3$, 10^6 , and 10^9 ? Assume that you have a sorting program that takes about $c_1 N \lg N$ seconds to sort a file of size N and a merging program that takes about $c_2(N + M)$ seconds to merge a file of size N with one of size M , with $c_1 \approx c_2$.

8.2 How does the strategy of using insertion sort for the whole file compare with the two methods postulated in Exercise 8.1? (Assume that the small file is random, so each insertion goes about halfway into the large file, and the running time is about $c_3 MN/2$, with c_3 approximately the same as the other constants.)

8.3 Describe what happens if you try to use Program 8.1 for an in-place merge, by using the call `merge(a, a, N/2, a+N/2, N-N/2)` for the keys `A E Q S U Y E I N O S T`.

- **8.4** Does Program 8.1, called as described in Exercise 8.3, produce proper output if and only if the two input subarrays are in sorted order? Prove your answer, or provide a counterexample.

8.2 Abstract In-place Merge

Although implementing a merge seems to require extra space, we still find the *abstraction* of an in-place merge useful in the implementations of sorting methods that we examine here. In our next implementation of merging, we shall emphasize this point by using the interface `merge(a, l, m, r)` to indicate that the merge subroutine will put the result of merging `a[l], ..., a[m]` and `a[m+1], ..., a[r]` into a single ordered file, leaving the result in `a[l], ..., a[r]`. We could implement this merge routine by first copying everything to an auxiliary array and then using the basic method of Program 8.1; instead we shall consider an improvement to that approach. Although the extra *space* for the auxiliary array seems to be a fixed practical cost, we shall consider in Section 8.4 further improvements that allow us to avoid the the extra *time* required to copy the array.

The second characteristic of the basic merge that is worthy of note is that the inner loop includes two tests to determine whether the ends of the two input arrays have been reached. Of course, these two tests usually fail, and the situation thus cries out for the use of sentinel keys to allow the tests to be removed. That is, if elements with a key value larger than those of all the other keys are added to the ends of the *a* and *aux* arrays, the tests can be removed, because, when the *a* (*b*) array is exhausted, the sentinel causes the next elements for the *c* array to be taken from the *b* (*a*) array until the merge is complete.

As we saw in Chapters 6 and 7, however, it is not always easy to use sentinels, either because it might not be easy to know the largest key value or because space might not be available conveniently. For merging, there is a simple remedy, which is illustrated in Figure 8.1. The method is based on the following idea: Given that we are resigned to copying the arrays to implement the in-place abstraction, we simply put the second array in reverse order when it is copied (at no extra cost), so that its associated pointer moves from right to left. This arrangement leads to the largest element—in whichever array it is—serving as sentinel for the other array. Program 8.2 is an efficient

```

      A R S T G I N
A R S T N I G
  R S T N I G A
    R S T N I   A G
      R S T N   A G I
        R S T   A G I N
          S T   A G I N R
            T   A G I N R S
              A G I N R S T

```

Figure 8.1
Merging without sentinels

To merge two ascending files, we copy into an auxiliary array, with the second file in reverse order immediately following the first. Then, we follow this simple rule: Move the left or right item, whichever has the smaller key, to the output. The largest key serves as a sentinel for the other file, no matter in which file the key is. This figure illustrates how the files **A R S T** and **G I N** are merged.

Program 8.2 Abstract in-place merge

This program merges without using sentinels by copying the second array into *aux* in reverse order back to back with the first (putting *aux* in *bitonic* order). The first *for* loop moves the first array and leaves *i* pointing to *l*, ready to begin the merge. The second *for* loop moves the second array, and leaves *j* pointing to *r*. Then, in the merge (the third *for* loop), the largest element serves as the sentinel in whichever array it is. The inner loop of this program is short (move to *aux*, compare, move back to *a*, increment *i* or *j*, increment and test *k*).

```
Item aux[maxN];
merge(Item a[], int l, int m, int r)
{ int i, j, k;
  for (i = m+1; i > l; i--) aux[i-1] = a[i-1];
  for (j = m; j < r; j++) aux[r+m-j] = a[j+1];
  for (k = l; k <= r; k++)
    if (less(aux[j], aux[i]))
      a[k] = aux[j--]; else a[k] = aux[i++];
}
```

implementation of the abstract in-place merge based on this idea; it serves as the basis for the sorting algorithms that we discuss later in this chapter. It still uses an auxiliary array of size proportional to the merge output, but it is more efficient than the straightforward implementation because it avoids the tests for the ends of the arrays.

A sequence of keys that increases, then decreases (or decreases, then increases) is referred to as a *bitonic* sequence. Sorting bitonic sequences is equivalent to merging, but it is sometimes convenient to cast a merging problem as a bitonic sorting problem; this method of avoiding sentinel tests is a simple example.

An important property of Program 8.1 is that the merge is stable: It preserves the relative order of duplicate keys. This characteristic is easy to verify, and it is often worth making sure that stability is maintained when we implement an abstract in-place merge, because a stable merge leads immediately to stable *sorting* methods, as we shall see in Section 8.3. It is not always easy to maintain stability: for example, Program 8.1 is not stable (see Exercise 8.6). This consideration further complicates the problem of developing a true in-place merge.

Exercises

- ▷ 8.5 Show how the keys AEQSUYEINOST are merged using Program 8.2, in the style of the example diagrammed in Figure 8.1.
- 8.6 Explain why Program 8.2 is not stable, and develop a version that is stable.
- 8.7 What is the result when Program 8.2 is used for the keys EASYQUESTION?
- 8.8 Does Program 8.2 produce proper output if and only if the two input subarrays are in sorted order? Prove your answer, or provide a counterexample.

8.3 Top-Down Mergesort

Once we have a merging procedure, it is not difficult to use that procedure as the basis for a recursive sorting procedure. To sort a given file, we divide it in half, recursively sort the two halves, and then merge them. An implementation is given in Program 8.3; an example is depicted in Figure 8.2. As mentioned in Chapter 5, this algorithm is one of the best-known examples of the utility of the *divide-and-conquer* paradigm for efficient algorithm design.

Program 8.3 Top-down mergesort

This basic mergesort implementation is a prototypical divide-and-conquer recursive program. It sorts the array $a[1], \dots, a[r]$ by dividing it into two parts $a[1], \dots, a[m]$ and $a[m+1], \dots, a[r]$, sorting them independently (via recursive calls), and merging the resulting ordered subfiles to produce the final ordered result. The *merge* function may need to use an auxiliary array big enough to hold a copy of the input, but it is convenient to consider the abstract operation as an in-place merge (see text).

```
void mergesort(Item a[], int l, int r)
{ int m = (r+1)/2;
  if (r <= 1) return;
  mergesort(a, l, m);
  mergesort(a, m+1, r);
  merge(a, l, m, r);
}
```

```

A S O R T I N G E X A M P L E
A S
O R
A O R S
I T
      G N
    G I N T
A G I N O R S T
      E X
        A M
        A E M X
          L P
          E L P
        A E E L M P X
A A E E G I L M N O P R S T X
```

Figure 8.2
Top-down mergesort example

Each line shows the result of a call on *merge* during top-down mergesort. First, we merge *A* and *S* to get *A S*; then, we merge *O* and *R* to get *O R*; then, we merge *O R* with *A S* to get *A O R S*. Later, we merge *I T* with *G N* to get *G I N T*, then merge this result with *A O R S* to get *A G I N O R S T*, and so on. The method recursively builds up small sorted files into larger ones.

Top-down mergesort is analogous to a top-down management style, where a manager gets an organization to take on a big task by dividing it into pieces to be solved independently by underlings. If each manager operates by simply dividing the given task in half, then putting together the solutions that the subordinates develop and passing the result up to a superior, the result is a process like mergesort. Not much real work gets done until someone with no subordinates gets a task (in this case, merging two files of size 1); but management does much of the work, putting together solutions.

Mergesort is important because it is a straightforward optimal sorting method (it runs in time proportional to $N \log N$) that can be implemented in a stable manner. These facts are relatively easy to prove.

As we have seen in Chapter 5 (and, for quicksort, in Chapter 7), we can use tree structures to help us to visualize the recursive call structure of a recursive algorithm, to help us to understand variants of the algorithm, and to expedite the analysis of the algorithm. For mergesort, the recursive call structure depends only upon the size of the input. For any given N , we define a tree, called a *divide-and-conquer tree*, that describes the sizes of the subfiles that are processed during the operation of Program 8.3 (see Exercise 5.73): If N is 1, the tree is a single node with label 1; otherwise, the tree is a node containing the file size N as the root, the tree for $\lfloor N/2 \rfloor$ as the left subtree, and the tree for $\lceil N/2 \rceil$ as the right subtree. Each node in the tree thus corresponds to a call on mergesort, with the label giving the problem size corresponding to the recursive call. When N is a power of 2, this construction leads to a complete balanced tree with powers of 2 in all the nodes and 1s in all the external nodes. When N is not a power of 2, the tree is more complicated. Examples of both cases are illustrated in Figure 8.3. We have encountered such trees before, when considering an algorithm with the same recursive call structure as mergesort, in Section 5.2.

Structural properties of divide-and-conquer trees are directly relevant to the analysis of mergesort. For example, the total number of comparisons used by the algorithm is precisely the sum of all the node labels.

Property 8.1 *Mergesort requires about $N \lg N$ comparisons to sort any file of N elements.*